AI Trading Journal: An AI-Powered Trading Journal and Performance Analysis Mobile Application



YOUR FULL NAME

BSCS/20XX/XXXXX

This project is submitted in partial fulfillment of the requirements for the award of Bachelor of Science in Computer Science.

Faculty of Computer Science
Umma University
Kajiado, Kenya
October, 2026

This thesis is available for library use on the understanding that it is copyright material and that no quotation from the thesis may be published without proper acknowledgment.

DECLARATION

I **YOUR FULL NAME** hereby declare that this document is entirely my own work and has not been submitted to any university or institution for the award of a degree. To the best of my knowledge, it does not include any material information authored or published by others except where due reference has been made.

Student's signature:

Name..... Signature.....

Date.....

This system project has been submitted for examination with my approval as university Supervisor.

Supervisor's signature:

Signature..... Date.....

SUPERVISOR NAME

Umma University,

Kenya

DEDICATION

This project is wholeheartedly dedicated first and foremost to Almighty Allah, by whose grace all things are possible. To my family, whose steadfast love, encouragement, and support have been my constant source of strength. I also extend my deepest gratitude to my lecturers and mentors, whose wisdom and guidance laid the groundwork for this achievement. To my friends and classmates who continued to encourage me, thank you. This work stands as a testament to discipline, determination, and the pursuit of excellence.

ACKNOWLEDGEMENT

I hereby express my gratitude to my supervisor, **SUPERVISOR NAME**, whose continuous feedback and guidance assisted in the successful completion of this project on time. I also extend heartfelt thanks to the traders, trading mentors and community administrators who generously shared their experiences and time during requirements gathering. I am grateful to the dedicated members of the Computer Science Department for their mentorship and technical insights; their knowledge of software engineering principles has significantly influenced the system's structure, security and performance.

This project continues to be a work in progress, and I remain committed to improving it further based on user feedback and future research opportunities.

ABSTRACT

Retail participation in the foreign exchange (forex) market has grown rapidly in Kenya since the Capital Markets Authority began licensing online forex brokers, yet regulators consistently report that the majority of retail trading accounts lose money. Research in behavioural finance attributes much of this underperformance to cognitive and emotional biases such as overconfidence, loss aversion and the disposition effect, compounded by weak risk management and the absence of structured self-review. Professional traders and trading psychologists recommend keeping a detailed trading journal, but most retail traders either keep no records or rely on notebooks and spreadsheets that are tedious to maintain and difficult to analyse.

This project presents the design and development of the **AI Trading Journal and Performance Analysis System**, a cross-platform mobile application that allows forex traders to record completed trades, track profit and loss, monitor win/loss and risk/reward statistics, set measurable goals, and receive AI-generated insights about their own historical trading behaviour. The system supports two roles: **Traders**, who manage a private journal, and **Administrators**, who monitor platform usage through aggregated, privacy-preserving statistics. The application is built with React Native, Expo SDK 57 and TypeScript on the front end, and Supabase (PostgreSQL with Row Level Security, Authentication, Storage and Edge Functions) on the back end. Google Gemini is integrated through a secure server-side Edge Function so that the AI key never reaches the mobile application; the model receives only de-identified journal data and returns structured insights covering strengths, weaknesses, trading patterns, risk-management observations, improvements and reflective questions.

Security was treated as a first-class requirement: validation on the client and in database constraints, rate limiting, email verification by one-time code, a Content Security Policy for the web build, role-based access enforced by Row Level Security, audit logging and least-privilege column grants. A suite of 38 automated security tests against the live backend passed in full. The system is explicitly a decision-support and performance-analysis tool; it does not execute trades or connect to brokers, and all AI output is presented as educational analysis rather than financial advice.

Key words: Trading Journal, Forex, Performance Analytics, Behavioural Finance, Artificial Intelligence, Google Gemini, React Native, Supabase, Row Level Security.

TABLE OF CONTENTS

Declaration ..... **ii**

Dedication ..... **iii**

Acknowledgement ..... **iv**

Abstract ..... **v**

1. INTRODUCTION **1**

1.1 Background of the Study 1

1.2 Problem Statement 2

1.3 Objectives 3

1.3.1 General Objective 3

1.3.2 Specific Objectives 3

1.4 Research Questions 3

1.5 Justification 3

1.6 Scope of the Project 4

1.7 Limitations 4

1.8 Summary 5

2. LITERATURE REVIEW **6**

2.1 Introduction 6

2.2 Challenges in Traditional Trade Journaling and Performance Review 6

2.2.1 Behavioural Biases in Retail Trading 6

2.2.2 Inefficiencies of Manual Journaling Methods 7

2.2.3 Risk Management and Emotional Discipline 7

2.3 Review of Existing Trading Journal Applications 7

2.3.1 Overview of Existing Trading Journal and Analytics Tools 7

2.3.2 Usability, Accessibility and Functional Limitations 8

2.3.3 Identified Gaps from Reviewed Systems 8

2.4 Development of an AI-Powered Trading Journal System 9

2.4.1 System Design and Architecture Overview 9

2.4.2 Implementation Using React Native, Expo and Supabase 9

2.4.3 Integration of AI Analysis Using Google Gemini 9

2.4.4 Expected Benefits to Traders 10

2.5 Testing of the AI Trading Journal Application 10

2.5.1 Functionality and Performance Testing 10

2.5.2 Usability and User Experience Evaluation 10
2.5.3 Security and Reliability Testing 11
2.6 Summary 11
3. SYSTEM DEVELOPMENT METHODOLOGY 12
3.1 Introduction 12
3.2 Methodological Approach 12
3.3 Agile Development Phases 13
3.3.1 Requirements Phase 13
3.3.2 Design Phase 14
3.3.3 Development Phase 14
3.3.4 Testing Phase 14
3.3.5 Deployment Phase 15
3.3.6 Review Phase 15
3.4 Summary 15
4. REQUIREMENTS GATHERING AND ANALYSIS 16
4.1 Introduction 16
4.2 Data Collection Approach 16
4.3 Data Collection Methods 16
4.4 Primary Data Collection 16
4.5 Data Analysis 17
4.5.1 Interview Data Summary 17
4.5.2 Questionnaire Analysis 18
4.6 Secondary Data Collection 23
4.7 Requirements and Stakeholders Identification 24
4.8 Challenges in Requirements Gathering and Analysis 25
4.9 Summary 26
5. SYSTEM DESIGN 27
5.1 Introduction 27
5.2 Structural Diagrams 27
5.2.1 System Architecture 27
5.2.2 Class Diagram 28
5.2.3 Entity Relationship Diagram (ERD) 29
5.3 Behavioural Diagrams 30
5.3.1 Use Case Diagram 30

5.3.2 Sequence Diagrams 31
5.3.3 Activity Diagram 33
5.4 Database Schema 35
5.5 Security Design 36
5.6 Wireframes 37
5.7 Summary 38
6. SYSTEM DEVELOPMENT 40
6.1 Introduction 40
6.2 System Development Tools and Techniques 40
6.2.1 Integrated Development Environment (IDE) 41
6.2.2 Front-end Development 42
6.2.3 Backend and Database Management Tools 42
6.2.4 AI Integration 42
6.3 System Interfaces 43
6.4 Challenges Encountered During Development 49
6.4.1 Technical Complexities 49
6.4.2 User Adoption and Resistance 49
6.4.3 Economic Constraints 49
6.4.4 Continuous Updates and Support 49
6.5 Summary 49
7. SYSTEM TESTING AND DEPLOYMENT 50
7.1 Introduction 50
7.2 System Testing and Validation 50
7.2.1 Unit and Module Tests 50
7.2.2 Functional Testing 51
7.2.3 User Interface (UI) Testing 51
7.2.4 Security Testing 52
7.3 Testing Results 53
7.4 System Deployment 53
7.4.1 Current Deployment Methods 54
7.4.2 Future Deployment Plans 54
7.5 Summary 54
8. DISCUSSIONS, FUTURE WORKS AND RECOMMENDATIONS 55
8.1 Introduction 55

8.2 Discussions of Objectives 55
8.3 Future Work 56
8.3.1 Deployment to App Stores 56
8.3.2 Broker Statement Import 56
8.3.3 Push Notifications and Reminders 56
8.3.4 Deeper AI Analysis 56
8.3.5 Localisation and Accessibility 56
8.4 Recommendations 56
8.4.1 Continuous User Engagement 56
8.4.2 Strengthening Data Security 56
8.4.3 Responsible Use of AI 56
8.4.4 Regular System Updates 57
8.4.5 Collaboration with Trading Communities and Regulators 57
8.5 Summary 57
Appendix **58**
A.1: User Survey Questionnaire 58
A.2: Work Plan and Schedule 59
A.3: Budget 60
A.4: My Project Repository 60
A.5: Plagiarism Report 62
References **63**

LIST OF FIGURES

Figure 3.1 Agile Methodology 13
Figure 4.1 Current Trade Recording Methods 19
Figure 4.2 Frequency of Performance Review 19
Figure 4.3 Influence of Emotions on Trading 20
Figure 4.4 Perceived Value of AI Insights 21
Figure 4.5 Mentor Review Methods 22
Figure 4.6 Common Weaknesses Observed by Mentors 22
Figure 4.7 Value of Aggregated Statistics 23
Figure 5.1 System Architecture 28
Figure 5.2 Class Diagram 29
Figure 5.3 Entity Relationship Diagram 30
Figure 5.4 Use Case Diagram 31
Figure 5.5 Sequence Diagram — AI Performance Analysis 32
Figure 5.6 Sequence Diagram — Registration and Email Verification 33
Figure 5.7 Activity Diagram 34
Figure 5.8 Database Schema 36
Figure 5.9 Wireframes 38
Figure 6.1 Visual Studio Code IDE 41
Figure 6.2 Application Running on a Physical Device (Expo Go) 41
Figure 6.3 Supabase Project Dashboard 42
Figure 6.4 Splash, Sign-in and Registration Screens 43
Figure 6.5 Email Verification and Password Reset 44
Figure 6.6 Trader Home Dashboard 44
Figure 6.7 Trading Journal and Add Trade 45
Figure 6.8 Trade Details and Analytics 45
Figure 6.9 AI Insights 46
Figure 6.10 Goals, Notifications and Profile 46
Figure 6.11 Settings and About 47
Figure 6.12 Administrator Dashboard and User Management 47
Figure 6.13 Administrator Monitoring and Reports 48
Figure 6.14 Administrator Settings and Profile 48
Figure 7.1 Security Test Suite Output (live backend) 53

LIST OF TABLES

Table 4.1 Primary Data Collection 17
Table 4.2 Retail Trader Interview Summary 17
Table 4.3 Trading Mentor Interview Summary 18
Table 4.4 Community Administrator Interview Summary 18
Table 4.5 Functional Requirements 24
Table 4.6 Non-Functional Requirements 25
Table 5.1 Security Controls 37
Table 6.1 Development Tools and Technologies 40
Table 7.1 Unit and Module Testing 50
Table 7.2 Functional Testing 51
Table 7.3 User Interface Testing 52
Table 7.4 Security Test Coverage 52
Table 7.5 Testing Results 53

CHAPTER ONE

1. INTRODUCTION

1.1 Background of the Study

The foreign exchange market is the largest financial market in the world. The Bank for International Settlements (2022) reported average turnover of about 7.5 trillion US dollars per day in its most recent Triennial Central Bank Survey. While the market was historically dominated by banks and institutions, internet brokers and mobile trading platforms have made it accessible to individual (retail) traders with small accounts, often through leveraged contracts for difference (CFDs).

Kenya has followed this global trend. The Capital Markets (Online Foreign Exchange Trading) Regulations of 2017 created a licensing framework for online forex brokers, and the Capital Markets Authority (CMA) has since licensed several non-dealing and money-manager brokers serving Kenyan clients. Combined with high smartphone penetration and mobile-money funding options, this has drawn many young Kenyans into retail forex trading, frequently through social-media communities and informal mentorship groups.

Despite its popularity, retail trading carries a high risk of loss. When the European Securities and Markets Authority (ESMA, 2018) introduced product-intervention measures on CFDs, it cited evidence that between 74% and 89% of retail investor accounts lose money on these products. Similar risk warnings are now mandatory on brokers' websites in many jurisdictions. Understanding *why* most retail traders lose is therefore an important practical question.

Behavioural finance offers much of the explanation. Prospect theory (Kahneman & Tversky, 1979) shows that people weigh losses more heavily than equivalent gains, which in trading manifests as holding losing positions too long and closing winners too early — the *disposition effect* described by Shefrin and Statman (1985) and documented empirically in brokerage data by Odean (1998). Barber and Odean (2000) found that the most active individual traders earned the lowest returns, attributing excessive trading to overconfidence. Lo, Repin and Steenbarger (2005) further observed that strong emotional reactions to gains and losses were associated with poorer trading performance among day traders.

Trading educators and psychologists consistently recommend a disciplined record-keeping process as the foundation of improvement. Elder (1993) describes the trading diary as essential for learning from one's own trades; Douglas (2000) emphasises objective self-assessment and

consistency; and Steenbarger (2003) frames performance improvement as a process of identifying one's own recurring patterns. A trading journal captures not only prices and profit or loss but also the strategy used, the trading session, the planned risk, the reason for entry, and the trader's emotional state — exactly the information needed to detect the biases described above.

In practice, however, most retail traders do not journal consistently. Paper notebooks and spreadsheets are tedious to maintain, broker statements contain only raw transactions without context, and even traders who keep records rarely analyse them systematically. At the same time, recent advances in large language models (LLMs) such as Google Gemini make it possible to summarise structured data and describe patterns in plain language. This creates an opportunity: a mobile journal that makes recording trades fast, computes performance statistics automatically, and uses AI to highlight each trader's own strengths and weaknesses — while keeping sensitive financial data private and secure.

1.2 Problem Statement

Retail forex traders in Kenya lack a simple, secure and affordable way to record their trades and learn from their own performance. Most traders either keep no records or rely on notebooks, spreadsheets and screenshots shared on messaging platforms. These methods make it difficult to calculate essential metrics such as win rate, profit factor, average risk/reward and losing streaks, and almost impossible to identify behavioural patterns such as overtrading, revenge trading after losses, or poorer results in particular sessions or emotional states.

Existing commercial trading-journal tools address parts of this problem but are largely desktop-oriented, priced in foreign currency on a subscription basis, and designed for markets and workflows outside the Kenyan context. Most provide statistics but leave the interpretation to the trader, and those that offer automated "insights" rarely explain them in plain language grounded in the trader's own notes. Mentors who coach groups of traders also lack a structured way to understand common weaknesses across their community without exposing individual traders' private data.

Consequently, traders repeat the same mistakes, risk management remains weak, and the high loss rates reported by regulators persist. There is a clear need for an integrated mobile system that combines structured journaling, automatic performance analytics and AI-generated, privacy-preserving performance insights in a single, secure application.

1.3 Objectives

1.3.1 General Objective

To develop an AI-powered Trading Journal and Performance Analysis mobile application that enables forex traders to record completed trades, analyse their performance through automatically computed statistics, and receive AI-generated insights into their trading behaviour, while keeping their data secure and private.

1.3.2 Specific Objectives

- To assess the challenges retail forex traders face in recording trades, reviewing performance and managing risk using traditional methods.
- To review existing trading-journal and performance-analytics applications and identify their limitations.
- To design and implement a secure mobile application for trade journaling, performance analytics and AI-powered insights using React Native (Expo), Supabase and Google Gemini.
- To test the functionality, usability and security of the AI Trading Journal application.

1.4 Research Questions

- What challenges do retail forex traders face when using traditional methods to record and review their trades?
- What are the limitations of existing trading-journal and analytics applications in terms of usability, accessibility, cost and depth of insight?
- How can an AI-powered trading journal be developed using React Native, Supabase and Google Gemini while protecting user data?
- How will the functionality, usability and security of the AI Trading Journal application be tested?

1.5 Justification

Retail traders lose money at high rates largely because of avoidable behavioural and risk-management mistakes, and the most widely recommended remedy — a disciplined trading journal — is rarely practised because existing methods are inconvenient. This project makes journaling fast on the device traders already use, computes the statistics that matter automatically, and uses AI to turn raw records into understandable, personalised feedback. By highlighting a

trader's own recurring patterns (for example, poor results in a particular session, or losses clustered after emotional trades), the system supports better decision-making without ever placing trades or giving guaranteed advice.

The project is also a meaningful application of core computer-science principles. It applies the full software development life cycle — requirements gathering, design, implementation, testing and deployment — and demonstrates modern practices in cloud back-ends, database security (Row Level Security), secure integration of AI services, and cross-platform mobile development.

1.6 Scope of the Project

The project delivers a cross-platform mobile application (Android and iOS, with a web build) called **AI Trading Journal**. It includes: secure email/password authentication with email verification; a trade journal with create, read, update and delete operations, search, filters and optional chart screenshots; a performance dashboard and analytics module with twelve key metrics and seven charts; trading goals that are tracked automatically from journal data; AI performance analysis of a selected period or a single trade using Google Gemini, with an analysis history; notifications; and profile, privacy and AI-data-sharing settings.

For administrators, the system provides a dashboard of aggregated platform statistics, trader account management (activate/deactivate), aggregated trading data, AI usage monitoring, exportable reports and system settings. The system does **not** execute trades, connect to broker accounts, handle payments or provide price forecasts; it is a journaling and decision-support tool only.

1.7 Limitations

The application depends on an internet connection for saving trades and requesting AI analyses; cached data can be viewed offline but changes require connectivity. The quality of AI insights depends on the quantity and honesty of the data the trader records, and the AI provider may occasionally be busy or rate-limited, which the system handles by falling back to alternative models. Profit and loss values are entered by the trader rather than imported from brokers, so their accuracy depends on the user. Some security features offered by the backend provider — such as leaked-password detection and point-in-time database recovery — require a paid plan and were therefore documented but not enabled. Finally, usability testing was carried out with a limited group of users, so large-scale behaviour has not yet been evaluated.

1.8 Summary

This chapter introduced the growth of retail forex trading in Kenya and the high loss rates associated with it, explained how behavioural biases and weak risk management contribute to these losses, and identified the lack of convenient, analytical and secure journaling tools as the core problem. It set out the general and specific objectives, research questions, justification, scope and limitations of the AI Trading Journal and Performance Analysis System, which forms the basis for the literature review in the next chapter.

CHAPTER TWO

2. LITERATURE REVIEW

2.1 Introduction

A literature review examines what researchers and developers have already done in a field, identifies what is known and where gaps remain, and provides the foundation for design decisions. This chapter reviews research on the behaviour and performance of retail traders, the role of journaling and self-review in improving performance, existing trading-journal applications, and the technologies used to build secure, AI-enabled mobile systems. It follows the study's objectives: it first examines the challenges of traditional journaling, then reviews existing applications and their gaps, explains how the proposed system is developed, and concludes with testing considerations.

2.2 Challenges in Traditional Trade Journaling and Performance Review

2.2.1 Behavioural Biases in Retail Trading

Behavioural finance research shows that individual traders systematically deviate from rational decision-making. Kahneman and Tversky's (1979) prospect theory established that losses are felt more strongly than equivalent gains and that people evaluate outcomes relative to a reference point. Applied to trading, this produces the disposition effect: the tendency to sell winning positions too early and hold losing positions too long (Shefrin & Statman, 1985). Odean (1998) confirmed this pattern empirically using thousands of brokerage accounts, showing that investors realised gains far more readily than losses.

Overconfidence is a second well-documented bias. Barber and Odean (2000) found that households that traded most frequently earned substantially lower net returns than those that traded least, and in later work (Barber & Odean, 2001) linked higher trading frequency to overconfidence. In the forex context, where leverage magnifies the cost of each mistake, overtrading and poor position sizing can quickly erode an account. Emotions also matter: Lo, Repin and Steenbarger (2005) reported that traders with more intense emotional reactions to monetary gains and losses tended to perform worse.

These studies show that many losses are caused by repeatable behavioural patterns rather than by random chance. Because such patterns are difficult to notice from memory alone, traders need objective records of their decisions — including emotional state and reasons for entry — to identify and correct them.

2.2.2 Inefficiencies of Manual Journaling Methods

Trading literature widely recommends keeping a journal. Elder (1993) describes a trading diary as a core tool for learning from both winning and losing trades, while Douglas (2000) argues that consistent success depends on objectively evaluating one's own behaviour rather than blaming the market. Steenbarger (2003) presents performance improvement as a process of identifying one's own patterns, strengths and weaknesses.

In practice, manual methods make this difficult. Paper notebooks cannot compute statistics; spreadsheets require formula knowledge and discipline to maintain; and screenshots shared in messaging groups are scattered and unstructured. Broker platforms such as MetaTrader provide account statements, but these list only transactions — they do not capture the strategy, session context, planned risk, reason for entry or emotional state that explain *why* a trade was taken. As a result, even traders who record their trades rarely convert those records into insight.

2.2.3 Risk Management and Emotional Discipline

Sound risk management — defining a stop loss, sizing positions consistently and seeking a favourable risk/reward ratio — is emphasised throughout trading education (Tharp, 2007). Regulators reinforce this: ESMA's (2018) intervention measures, including leverage limits and negative-balance protection, were motivated by evidence of widespread retail losses on leveraged products. Yet without records, traders cannot easily measure whether they actually follow their risk rules, how often they trade without a stop loss, or whether their losses are larger than their wins. A journal that automatically computes metrics such as average risk/reward, profit factor, largest loss and losing streaks makes these behaviours visible.

2.3 Review of Existing Trading Journal Applications

2.3.1 Overview of Existing Trading Journal and Analytics Tools

Several commercial tools support trade journaling and performance analysis. **Edgewonk** and **TraderSync** offer detailed statistics, tagging of setups and mistakes, and trade replays. **Tradervue** provides trade import from brokers and sharing of trades with other users. **Myfxbook** connects to forex trading accounts to track performance and publish verified statistics. Trading platforms such as **MetaTrader 4/5** provide account history and basic reports, while many retail traders fall back on **spreadsheets** and generic note-taking applications.

These tools demonstrate the value of structured trade data and automatic statistics. Some newer products also advertise automated insights. However, their primary focus is on data collection and charting; interpretation is largely left to the trader, and few explain patterns in plain language grounded in the trader's own written reasons and emotions.

2.3.2 Usability, Accessibility and Functional Limitations

Many existing tools are designed primarily for desktop use, with dense interfaces aimed at experienced traders. Nielsen's (1994) usability heuristics stress simplicity, recognition rather than recall and clear feedback — qualities that are harder to achieve in complex desktop analytics dashboards used on small screens. Most commercial journals are priced as monthly subscriptions in US dollars, which is a significant barrier for students and small retail traders in Kenya. Tools that rely on broker connections may not support all brokers used locally, and connecting a trading account introduces additional privacy and security concerns.

From a functional perspective, broker-connected tools capture transactions but not context (strategy, reasons, emotions), while manual tools capture context but provide little analysis. Few tools give mentors or community administrators aggregated, anonymised insights without exposing individual traders' private records.

2.3.3 Identified Gaps from Reviewed Systems

- **Limited interpretation:** most tools report statistics but do not explain, in plain language, what a specific trader should review based on their own data.
- **Weak capture of behavioural context:** broker-based statistics omit reasons for entry and emotional state, which are central to behavioural patterns.
- **Cost and accessibility:** subscription pricing and desktop-first design limit adoption among Kenyan retail traders who mainly use smartphones.
- **Privacy:** few tools combine AI analysis with explicit user control over what data is shared, de-identification before AI processing, and strict per-user data isolation.
- **Aggregated oversight:** mentors and administrators lack privacy-preserving, aggregated views of community behaviour and system usage.

2.4 Development of an AI-Powered Trading Journal System

2.4.1 System Design and Architecture Overview

The proposed system follows a layered client–server architecture. The **presentation layer** is a React Native application built with Expo and TypeScript, using Expo Router for navigation and React Native Paper for a consistent Material Design interface. The **application layer** consists of an API client abstraction, React Query for caching and synchronisation, and a statistics engine that computes performance metrics on the device. The **data layer** is a Supabase PostgreSQL database protected by Row Level Security (RLS), with Supabase Storage for chart screenshots and Supabase Auth for identity. The **AI layer** is a Supabase Edge Function that calls the Google Gemini API server-side, so that the AI key is never exposed in the mobile application.

A key design principle is that the client is never trusted: every rule enforced in the user interface (validation, ownership, roles and rate limits) is enforced again by the database or Edge Function. This follows the OWASP Mobile Top 10 guidance on insecure authentication, insecure data storage and insufficient input validation (OWASP Foundation, 2024).

2.4.2 Implementation Using React Native, Expo and Supabase

React Native allows a single TypeScript codebase to produce native Android and iOS applications as well as a web build. Expo provides a managed toolchain — including Expo Go for testing on physical devices, over-the-air updates and cloud builds — which reduces the cost and complexity of cross-platform development. Form handling uses React Hook Form with Zod schemas for type-safe validation, and sensitive session tokens are stored in the device keychain or keystore through Expo SecureStore.

Supabase is an open-source backend platform built on PostgreSQL. Instead of writing a custom API server, the application communicates with PostgreSQL through Supabase's auto-generated REST interface, and access is governed by RLS policies written in SQL. This moves authorisation into the database itself, where it cannot be bypassed by a modified client. Supabase Auth issues short-lived JSON Web Tokens with refresh-token rotation, and Storage buckets enforce size and file-type limits and per-user folder policies.

2.4.3 Integration of AI Analysis Using Google Gemini

Large language models can summarise structured data and describe patterns in natural language. The system integrates Google Gemini through an Edge Function that verifies the caller's session, loads only that user's trades through RLS, computes summary statistics, and sends Gemini a de-identified payload containing no names, emails or identifiers. The trader controls whether free-

text notes and emotional states are included. Gemini is instructed to base observations only on the supplied data, to distinguish observations from suggestions, and to avoid presenting anything as guaranteed financial advice. The response is constrained to a JSON schema with sections for a performance summary, strengths, weaknesses, trading patterns, risk-management observations, areas for improvement and reflective questions, and is validated and sanitised before being stored and displayed.

2.4.4 Expected Benefits to Traders

Traders gain a fast way to journal on their phones, automatic calculation of the statistics that matter, and understandable feedback on their own behaviour — for example, discovering that a particular strategy or session is consistently unprofitable, that losses cluster after emotional trades, or that many trades lack a stop loss. Goals such as “stop overtrading” or “maintain an average risk/reward above 1.8” are tracked automatically. Administrators gain visibility of platform usage and aggregated trading behaviour without accessing private journal entries, supporting responsible, privacy-respecting community management.

2.5 Testing of the AI Trading Journal Application

2.5.1 Functionality and Performance Testing

Functional testing verifies that each feature works according to the requirements: registration and verification, sign-in, creating, editing and deleting trades, analytics calculations, goal tracking, AI analysis and administrative functions. Integration testing confirms that the modules work together — for example, that a newly recorded trade immediately updates the dashboard, analytics and goal progress. Performance testing considers the latency of AI requests, which depend on an external provider, and verifies that the system degrades gracefully when the provider is busy.

2.5.2 Usability and User Experience Evaluation

Usability evaluation assesses whether traders with varying experience can record trades and interpret results without assistance, guided by established heuristics such as visibility of system status, consistency and error prevention (Nielsen, 1994). Particular attention is given to clear BUY/SELL indicators, readable charts, informative empty and error states, and support for both light and dark themes.

2.5.3 Security and Reliability Testing

Because the system stores personal financial data, security testing is essential. Tests attempt to access or modify another user's data, escalate privileges, forge AI results, call administrative functions without authorisation, and submit invalid data, verifying that each attempt is rejected by the backend. The OWASP Application Security Verification Standard (OWASP Foundation, 2021) provides a reference for authentication, access-control and validation requirements. Reliability testing verifies behaviour when the network is unavailable, when sessions expire, and when the AI provider returns errors.

2.6 Summary

This chapter reviewed evidence that retail traders commonly suffer from behavioural biases such as loss aversion, the disposition effect and overconfidence, and that disciplined journaling is widely recommended but rarely practised because manual methods are inconvenient. Existing journal tools provide statistics but are often desktop-oriented, costly, weak at capturing behavioural context and limited in plain-language interpretation. The review motivates an integrated, mobile-first, AI-assisted and privacy-preserving trading journal, and outlined how the proposed system will be designed, implemented with React Native, Supabase and Gemini, and tested.

CHAPTER THREE

3. SYSTEM DEVELOPMENT METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted in developing the AI Trading Journal and Performance Analysis System. It describes the software development approach and the phases followed in creating a functional, secure product. Because the system combines a mobile interface, a cloud database, security policies and an external AI service, a flexible and iterative approach was required so that each part could be built, tested and refined in turn. For this reason, the Agile software development methodology was adopted.

3.2 Methodological Approach

Agile emphasises working software, collaboration and responding to change over rigid, up-front planning (Beck et al., 2001). Development is divided into short iterations, or sprints, each producing a usable increment that is reviewed before the next begins (Schwaber & Sutherland, 2020). Unlike the Waterfall model, Agile allows requirements to evolve as feedback is received (Sommerville, 2016).

Agile suited this project well. The front end was first built against a mock API that mirrored the planned backend contract, allowing the user interface to be reviewed early. The real backend was then connected without changing any screen code. When problems were discovered during testing — for example, an AI model that was no longer available to new API keys — they were fixed in the next iteration rather than requiring a redesign.

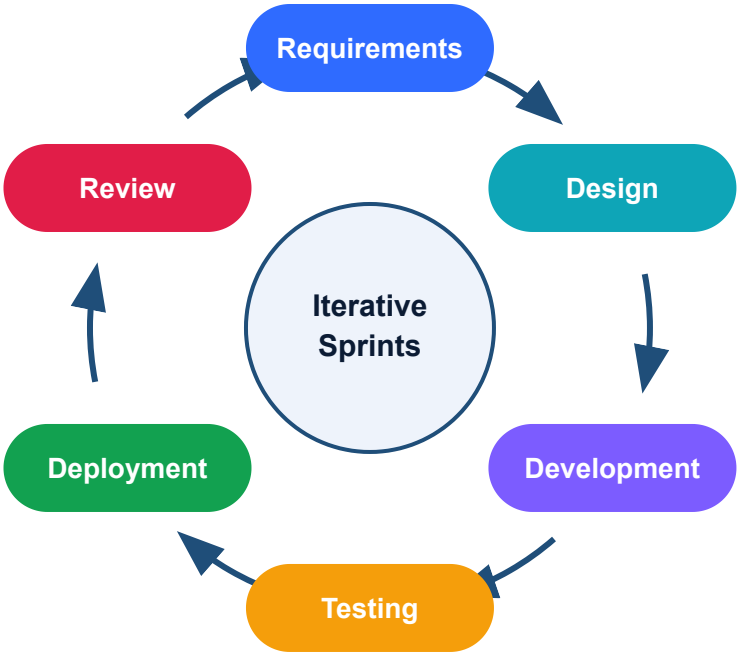


Figure 3.1 Agile Methodology

3.3 Agile Development Phases

The development of the AI Trading Journal followed six Agile phases: requirements, design, development, testing, deployment and review. Each phase contributed incrementally to achieving the project objectives.

3.3.1 Requirements Phase

Requirements were gathered through interviews and questionnaires with retail traders, trading mentors and community administrators, together with a review of literature and existing journal applications (Chapter Four). The requirements were organised into modules: authentication and profile; trade journal; analytics and goals; AI insights; notifications and settings; and administration. Security requirements — validation, rate limiting, content security policy, caching, monitoring, database-level security, authentication, logging, backups and connection pooling — were defined from the start rather than added later.

3.3.2 Design Phase

The design phase translated requirements into architecture, data and interface models: the system architecture, class and entity-relationship diagrams, use-case, sequence and activity diagrams, the database schema with its security policies, and low-fidelity wireframes (Chapter Five). A key design decision was to define a single API contract used by both a mock implementation and the real Supabase implementation, so that the front end could be completed and tested before the backend existed.

3.3.3 Development Phase

Development proceeded in four sprints:

- **Sprint 1 — Front end with mock API:** project setup with Expo SDK 57, branding and logo, navigation, all trader and admin screens, charts, forms with validation, and a mock backend that enforced the same rules as the planned server.
- **Sprint 2 — Backend:** PostgreSQL schema, constraints, indexes, triggers, Row Level Security policies, storage buckets, administrator functions and authentication configuration on Supabase.
- **Sprint 3 — AI integration:** the Gemini Edge Function with quotas, rate limits, de-identified payloads, response validation and model fallback.
- **Sprint 4 — Integration, testing and hardening:** switching the app to the live backend, email verification by one-time code, automated security and end-to-end tests, and fixes for the defects they revealed.

3.3.4 Testing Phase

Testing was performed at the end of every sprint and included:

- **Static checks:** TypeScript type-checking, ESLint and Expo Doctor on every change.
- **Module tests:** verifying individual rules such as database constraints, password policy and Edge Function input validation.
- **Functional and end-to-end tests:** automated browser runs of complete user journeys against the live backend.
- **Security tests:** 38 automated checks attempting cross-user access, privilege escalation and forged data.
- **User-interface tests:** visual review of every screen in light and dark themes at phone size.

3.3.5 Deployment Phase

The backend was deployed to a Supabase cloud project (database migrations, storage, authentication settings and the Edge Function). The mobile application is distributed for testing through Expo Go, and can be packaged for Android and iOS through Expo Application Services (EAS). Secrets such as the Gemini API key are stored only as server-side Edge Function secrets.

3.3.6 Review Phase

After each sprint, results were reviewed and improvements identified. Examples include moving the administrator logout into a dedicated Profile tab after users could not find it, shortening the home-screen greeting so that long names are not truncated, replacing email links with six-digit verification codes that work reliably on mobile devices, and adding automatic fallback between AI models when the primary model was overloaded.

3.4 Summary

The Agile methodology was adopted because it supports iterative delivery, early feedback and continuous improvement. Through six phases and four sprints, the project progressed from requirements to a secure, tested system, with a mock-first approach that allowed the interface to be validated before the backend was connected.

CHAPTER FOUR

4. REQUIREMENTS GATHERING AND ANALYSIS

4.1 Introduction

This chapter outlines the process of gathering and analysing requirements for the AI Trading Journal and Performance Analysis System. The objective was to identify the needs of the key users — retail forex traders, trading mentors and community administrators — in order to design a practical, user-centred and secure mobile solution. Requirements were collected through interviews, questionnaires and a review of literature and existing applications, and were analysed to derive functional and non-functional requirements.

4.2 Data Collection Approach

A mixed approach was adopted. Qualitative data from interviews and informal discussions provided detailed descriptions of how traders record and review trades, the difficulties they face and the features they expect. Quantitative data from short questionnaires provided simple distributions that indicate how common these practices and challenges are. Combining both approaches allowed the findings to be cross-checked.

4.3 Data Collection Methods

Both primary and secondary data collection methods were used. The main techniques were:

- Semi-structured interviews with retail traders, trading mentors and community administrators.
- Short questionnaires distributed to the same groups.
- Observation of how traders currently share and discuss trades in online trading communities.
- Review of research literature, regulatory publications and existing trading-journal applications.

4.4 Primary Data Collection

Primary data collection focused on obtaining direct insights from people involved in retail trading. A total of 45 participants took part in the study, comprising:

Stakeholder	Number of Respondents
Retail Forex Traders	30
Trading Mentors / Coaches	10
Trading Community Administrators	5
Total	**45**

Table 4.1 Primary Data Collection

Interviews captured participants' experiences with recording and reviewing trades, managing risk and emotions, and learning from mistakes. Questionnaires gathered simple counts on current practices and on the perceived value of automatic statistics and AI insights.

4.5 Data Analysis

Quantitative questionnaire data were summarised using counts and percentages and presented as pie charts. Qualitative interview data were analysed thematically to identify recurring challenges and expectations.

4.5.1 Interview Data Summary

The following tables summarise the main themes that emerged from the interviews.

Theme	Key Insights from Traders
Record keeping	Most do not record trades consistently; those who do use notebooks, spreadsheets or screenshots that are hard to analyse.
Performance review	Rarely calculate win rate, profit factor or risk/reward; rely on memory and account balance.
Emotions	Fear, greed and revenge trading after losses frequently influence decisions.
Risk management	Stop losses are sometimes skipped or moved; position sizes vary widely.
Expectations	A fast mobile journal with automatic statistics and personalised feedback in plain language, with data kept private.

Table 4.2 Retail Trader Interview Summary

Theme	Key Insights from Mentors
Reviewing students	Reviews are based on screenshots and verbal accounts, which are incomplete and inconsistent.
Common weaknesses	Poor risk management, overtrading and emotional decision-making are the most frequent problems.
Data quality	Students rarely record reasons for entry or emotional state, which are essential for coaching.
Expectations	Structured journals that capture context, and summaries that highlight recurring patterns.

Table 4.3 Trading Mentor Interview Summary

Theme	Key Insights from Administrators
Visibility	No reliable view of how many members actively journal or which strategies and pairs are popular.
Privacy	Members are unwilling to share individual records; aggregated information is acceptable.
Account management	Need to deactivate abusive or inactive accounts and adjust platform limits.
Expectations	A dashboard of aggregated statistics, AI usage monitoring and exportable reports.

Table 4.4 Community Administrator Interview Summary

4.5.2 Questionnaire Analysis

Retail Trader Questionnaire Analysis

A total of 30 retail traders completed the questionnaire. Its objective was to understand how traders currently record and review their trades and how they perceive AI-assisted analysis.

Question 1: How do you currently record your trades?

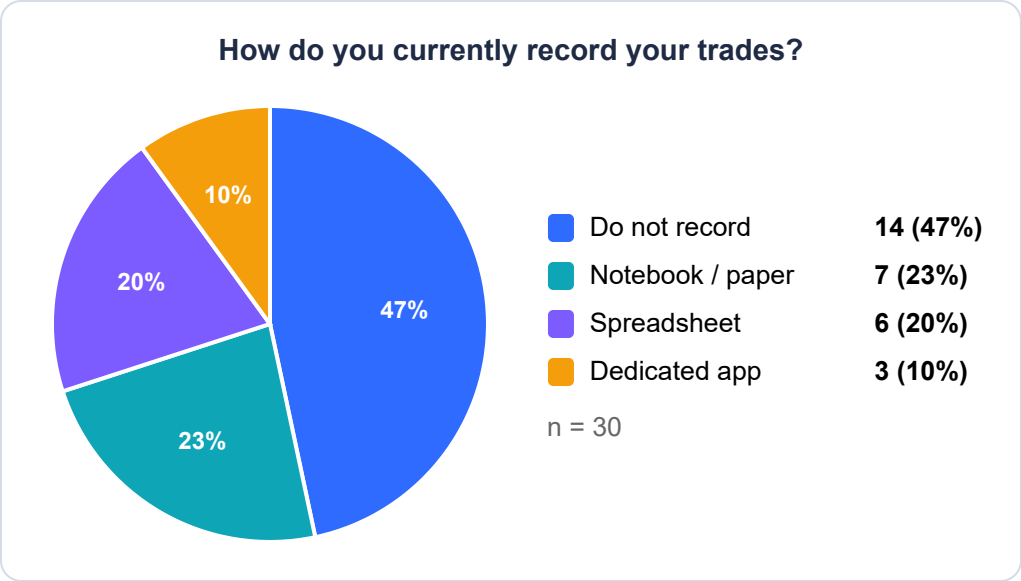


Figure 4.1 Current Trade Recording Methods

Analysis: The most common response was "Do not record" (47% of respondents), indicating a clear need for support in this area.

System Solution: The system provides a fast mobile trade form with required fields, automatic risk/reward and P/L suggestions, and optional chart screenshots, making consistent journaling practical.

Question 2: How often do you review your past trading performance?

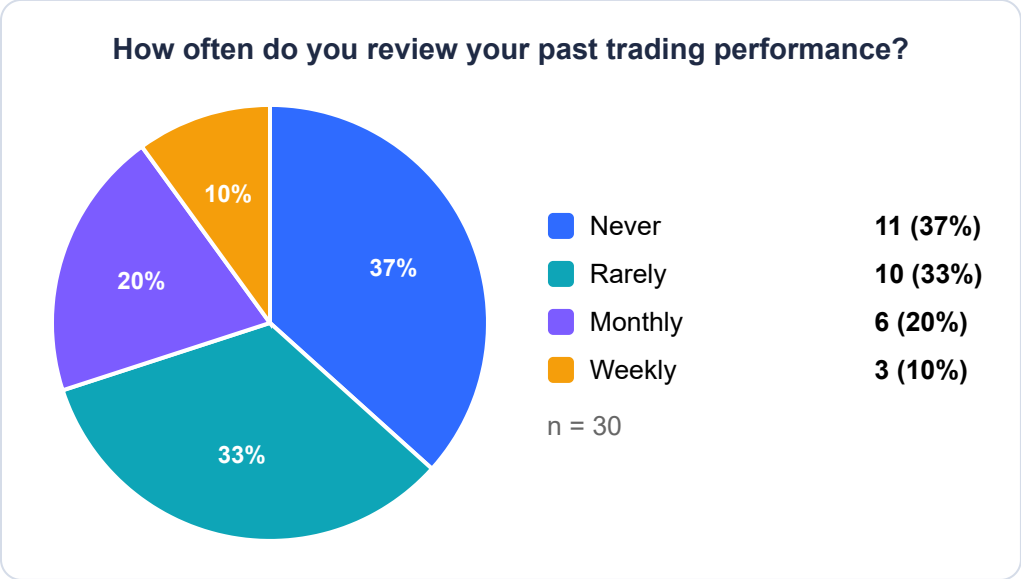


Figure 4.2 Frequency of Performance Review

Analysis: The most common response was “Never” (37% of respondents), indicating a clear need for support in this area.

System Solution: The Home dashboard and Analytics module compute performance statistics automatically every time a trade is saved, so review requires no manual calculation.

Question 3: Do emotions (fear, greed, stress) affect your trading decisions?



Figure 4.3 Influence of Emotions on Trading

Analysis: The most common response was “Often” (53% of respondents), indicating a clear need for support in this area.

System Solution: Each trade records an optional emotional state; analytics and AI insights compare results across emotional states to reveal their impact.

Question 4: Would AI-generated insights on your own journal help you improve?

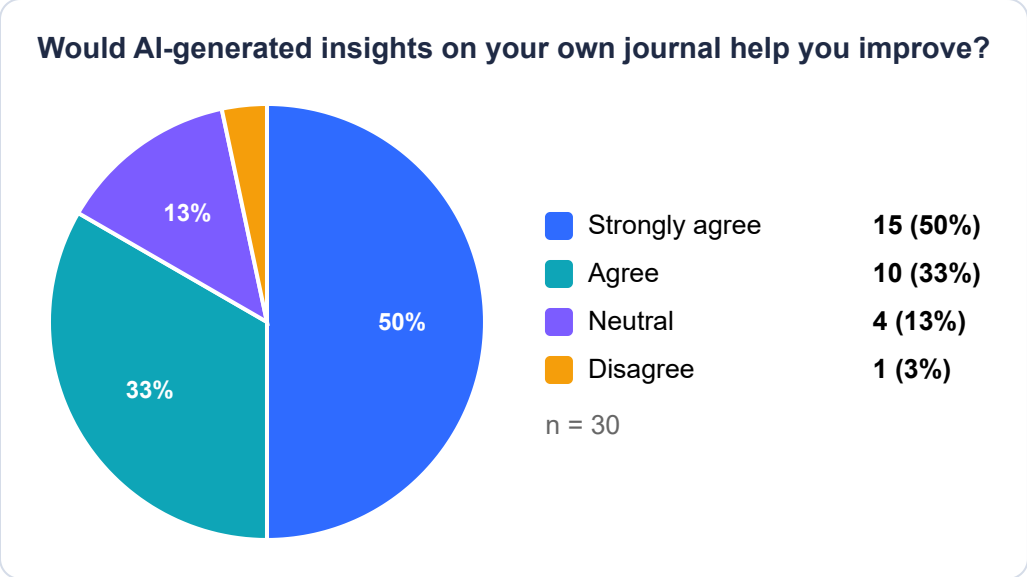


Figure 4.4 Perceived Value of AI Insights

Analysis: The most common response was “Strongly agree” (50% of respondents), indicating a clear need for support in this area.

System Solution: The AI Insights module analyses the trader’s own journal and presents strengths, weaknesses, patterns and suggested questions in plain language.

Trading Mentor Questionnaire Analysis

A total of 10 trading mentors completed the questionnaire, which focused on how they review students’ trading and the weaknesses they observe.

Question 1: How do you currently review your students' trading records?

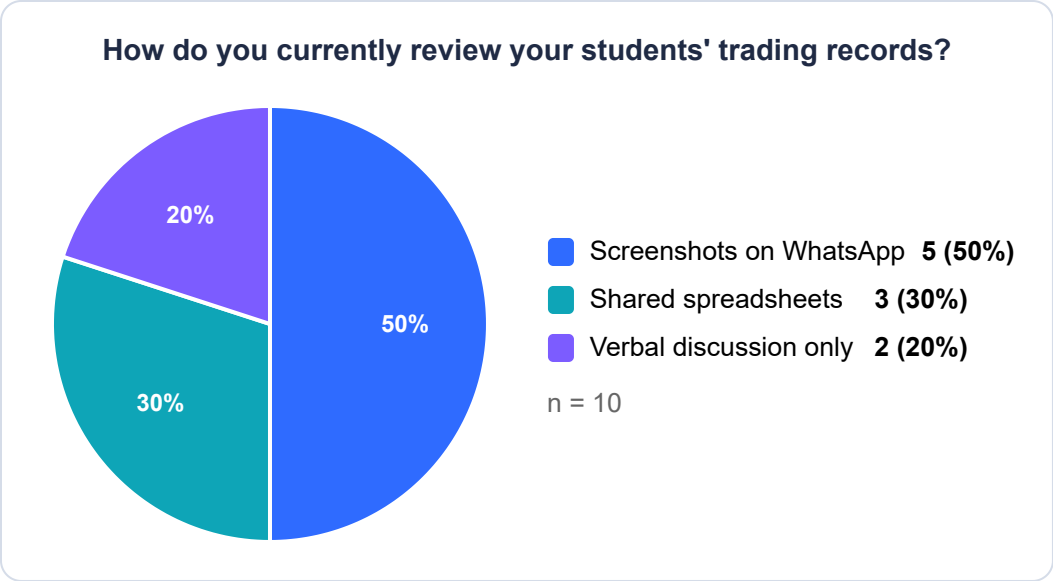


Figure 4.5 Mentor Review Methods

System Solution: Structured trade records with strategy, session, reasons and emotions give a complete picture that screenshots alone cannot provide.

Question 2: What is the most common weakness you observe in retail traders?



Figure 4.6 Common Weaknesses Observed by Mentors

System Solution: Analytics highlight risk/reward, losing streaks and trades without stop losses, while goals such as “maximum trades per day” directly target overtrading.

Community Administrator Questionnaire Analysis

Question 1: Would aggregated platform statistics help you support your community?

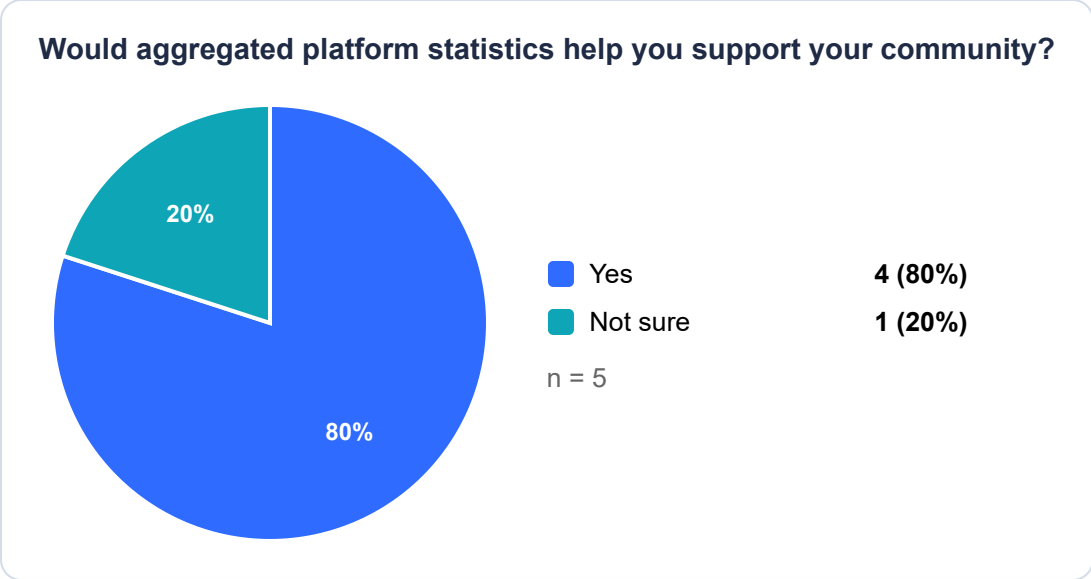


Figure 4.7 Value of Aggregated Statistics

System Solution: The administrator dashboard provides aggregated statistics, AI usage monitoring and reports while never exposing individual journal entries.

4.6 Secondary Data Collection

Secondary data complemented the primary findings. Regulatory publications (ESMA, 2018) and market data (Bank for International Settlements, 2022) established the scale of retail participation and losses, while the Capital Markets (Online Foreign Exchange Trading) Regulations (2017) provided the Kenyan regulatory context. Behavioural-finance research (Kahneman & Tversky, 1979; Shefrin & Statman, 1985; Odean, 1998; Barber & Odean, 2000) explained the patterns traders described in interviews. Documentation for React Native, Expo, Supabase and Google Gemini, together with OWASP security guidance, informed the technical and security requirements.

4.7 Requirements and Stakeholders Identification

Functional Requirements

ID	Requirement
FR01	The system shall allow users to register with email and password and verify their email with a six-digit code.
FR02	The system shall authenticate users and route them to the Trader or Admin interface according to their role.
FR03	Traders shall create, view, edit and delete their own completed trades, including an optional chart screenshot.
FR04	Traders shall search and filter trades by pair, strategy, direction, result, session and date.
FR05	The system shall compute performance metrics and charts dynamically from journal data.
FR06	Traders shall create goals whose progress is tracked automatically from their trades.
FR07	Traders shall request AI analysis of a period or a single trade and view previous analyses.
FR08	Traders shall control whether notes and emotions are shared with the AI analyst.
FR09	Administrators shall view aggregated platform statistics, trading data and AI usage.
FR10	Administrators shall activate or deactivate trader accounts (but not other administrators).
FR11	Administrators shall generate and export reports and manage system settings.
FR12	The system shall provide in-app notifications, password reset, and profile management.

Table 4.5 Functional Requirements

Non-Functional Requirements

Category	Requirement
Security	Row Level Security on all data; AI key only on the server; secure token storage; validation on client and server; rate limiting; audit logging.
Privacy	Traders see only their own data; administrators see only aggregates; AI receives de-identified data.
Performance	Cached data shown instantly with background refresh; AI analysis completes within the provider's time limits.
Usability	Clear, consistent interface with loading, empty and error states and light/dark themes.
Reliability	Graceful handling of offline use, expired sessions and AI provider errors.
Maintainability	Typed codebase, a single API contract, versioned SQL migrations and documented deployment.
Compliance	Clear disclaimer that AI output is educational and not financial advice; no trade execution.

Table 4.6 Non-Functional Requirements

Stakeholders Identification

Retail traders are the primary users; they record trades, review analytics, set goals and request AI insights. **Administrators** operate the platform, manage accounts and settings, and monitor aggregated usage. **Trading mentors** benefit indirectly through students who bring structured journals and AI summaries to coaching sessions. **The AI provider** (Google) processes de-identified data under the platform's instructions, and **regulators** such as the Capital Markets Authority shape the context by requiring clear risk warnings and discouraging misleading claims.

4.8 Challenges in Requirements Gathering and Analysis

Sensitivity of financial information: some traders were reluctant to share their results. This was addressed by assuring confidentiality and focusing questions on practices rather than account sizes. **Varying experience levels:** beginners and experienced traders had different expectations; the design therefore keeps core flows simple while offering deeper analytics for those who want them. **Expectations of AI:** some participants expected price predictions; the scope was clarified to performance analysis of past trades only. **Technical constraints:** requested features such as automatic broker import were deferred to future work to keep the scope achievable.

4.9 Summary

This chapter described how requirements were gathered from traders, mentors and administrators through interviews, questionnaires and secondary sources. The findings showed inconsistent record keeping, little systematic performance review, strong emotional influences on trading, and demand for automatic statistics and personalised, private AI feedback. These findings were translated into twelve functional requirements and a set of non-functional requirements that guide the system design.

CHAPTER FIVE

5. SYSTEM DESIGN

5.1 Introduction

This chapter presents the design of the AI Trading Journal and Performance Analysis System. It describes the system architecture, the structural and behavioural UML models, the database schema and its security design, and the wireframes that guided interface development. Special attention is given to role-based access for traders and administrators, secure integration of the AI service, and database-level enforcement of security rules.

5.2 Structural Diagrams

Structural diagrams describe the static architecture of the system: its components, classes and data entities, and the relationships between them.

5.2.1 System Architecture

The architecture is organised into a mobile client, the Supabase platform and the external AI provider. The client never holds secrets beyond the public anon key; all authorisation happens in the database or Edge Function.

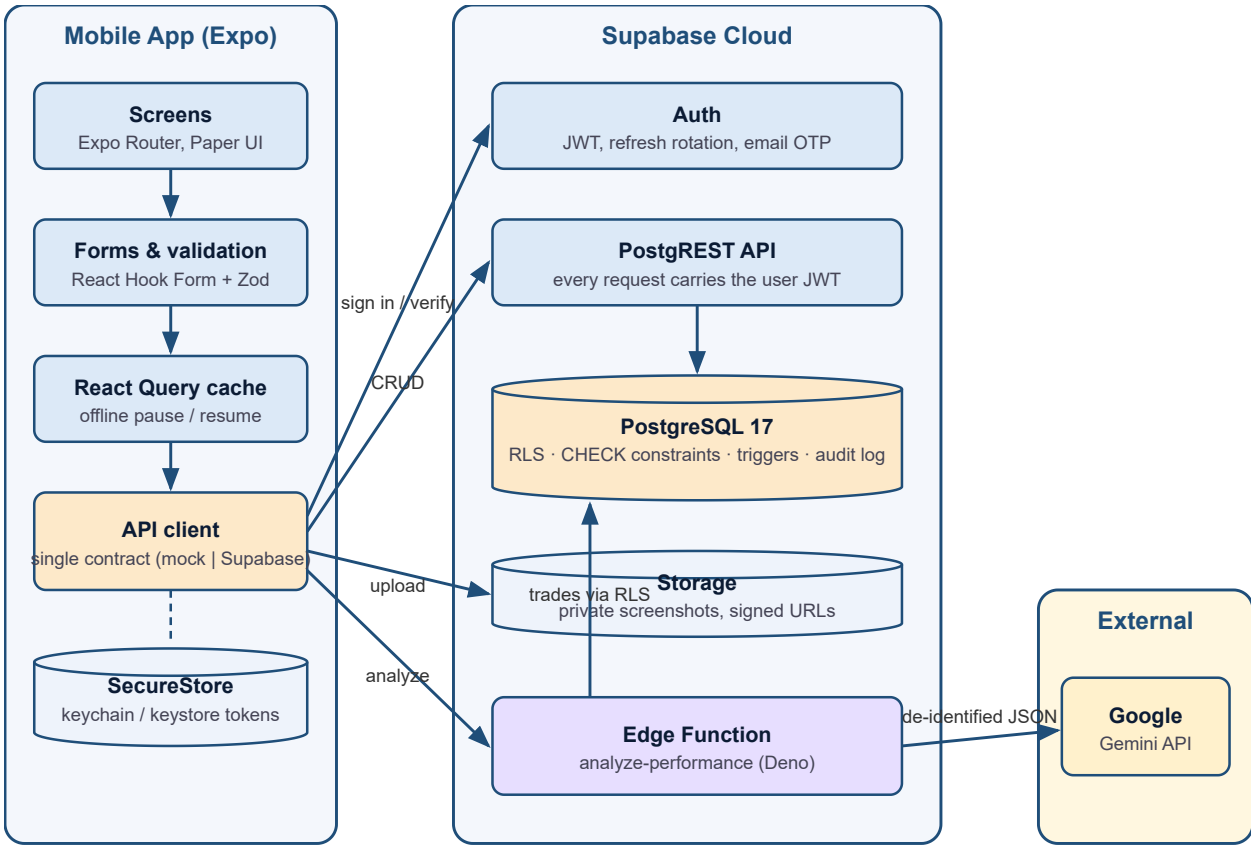


Figure 5.1 System Architecture

5.2.2 Class Diagram

The class diagram shows the main domain classes, their key attributes and operations, and the relationships between them.

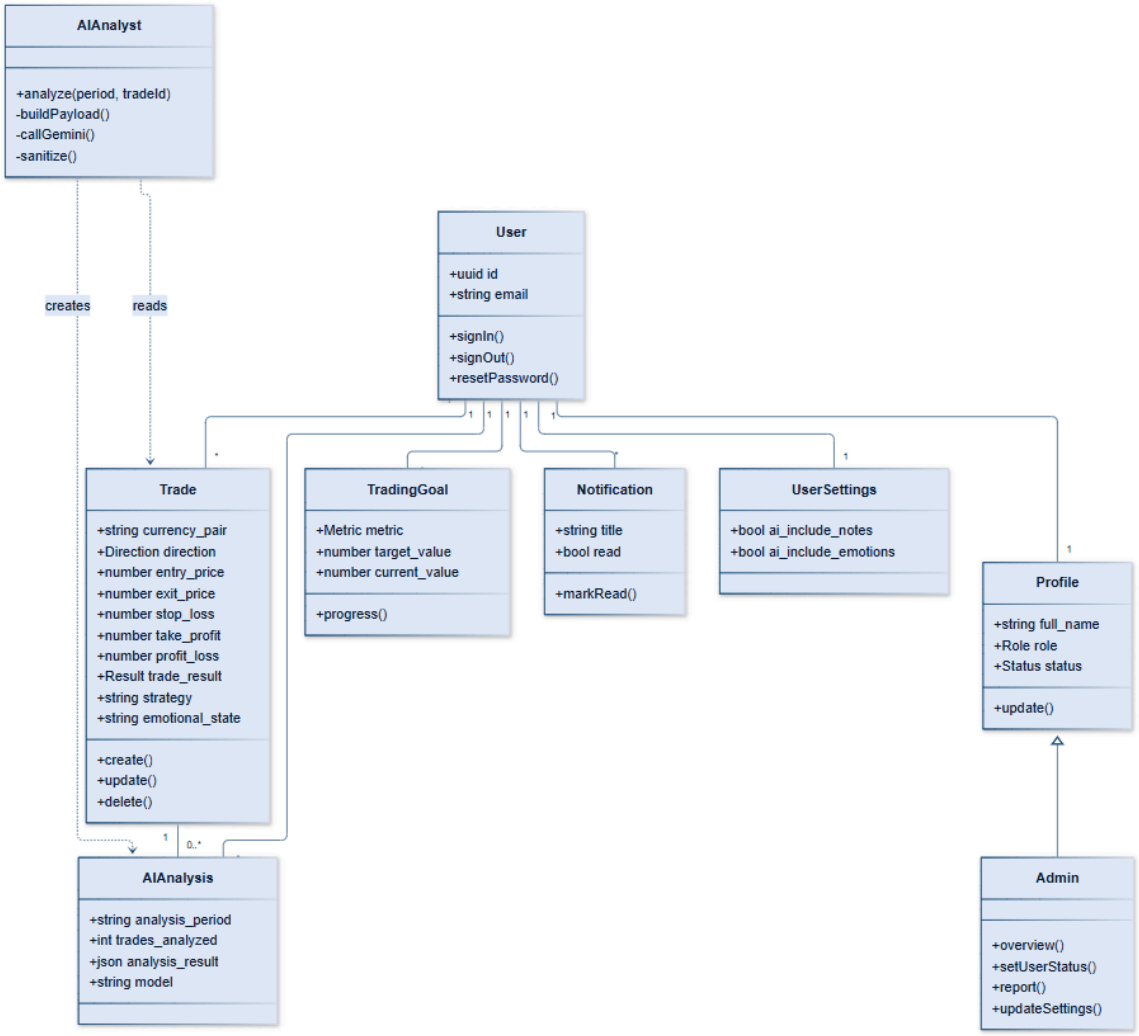


Figure 5.2 Class Diagram

5.2.3 Entity Relationship Diagram (ERD)

The ERD shows the database entities and their relationships. All user-owned tables reference the authentication users table and are deleted automatically when a user account is removed.

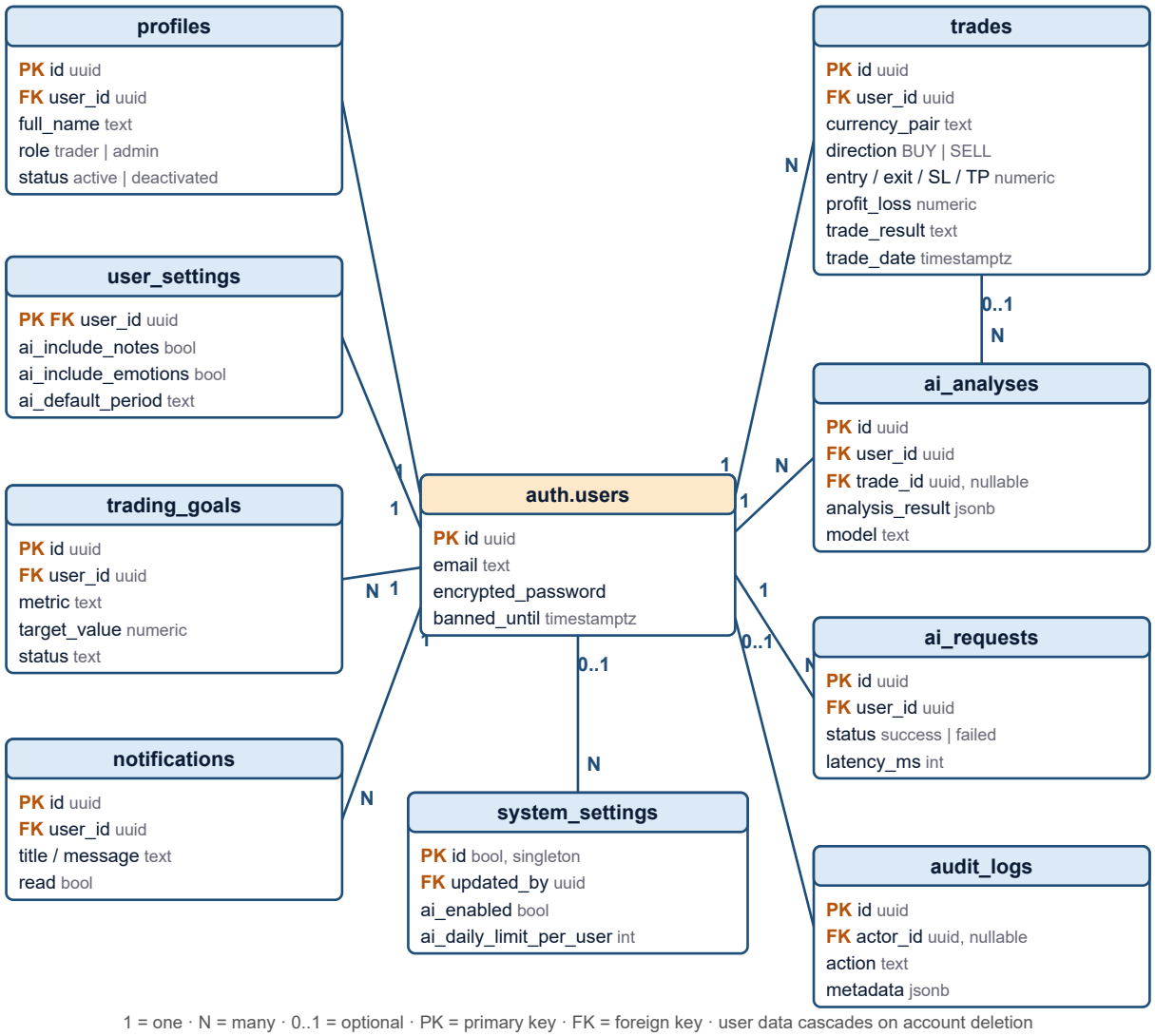


Figure 5.3 Entity Relationship Diagram

5.3 Behavioural Diagrams

Behavioural diagrams show how the system responds to user actions over time: who can do what (use cases), the order of interactions between components (sequence diagrams) and the flow of activities (activity diagram).

5.3.1 Use Case Diagram

The system has two human actors — the Trader and the Administrator — and one external system actor, the Gemini AI service. Traders manage their journal and request analyses; administrators manage the platform using aggregated data only.

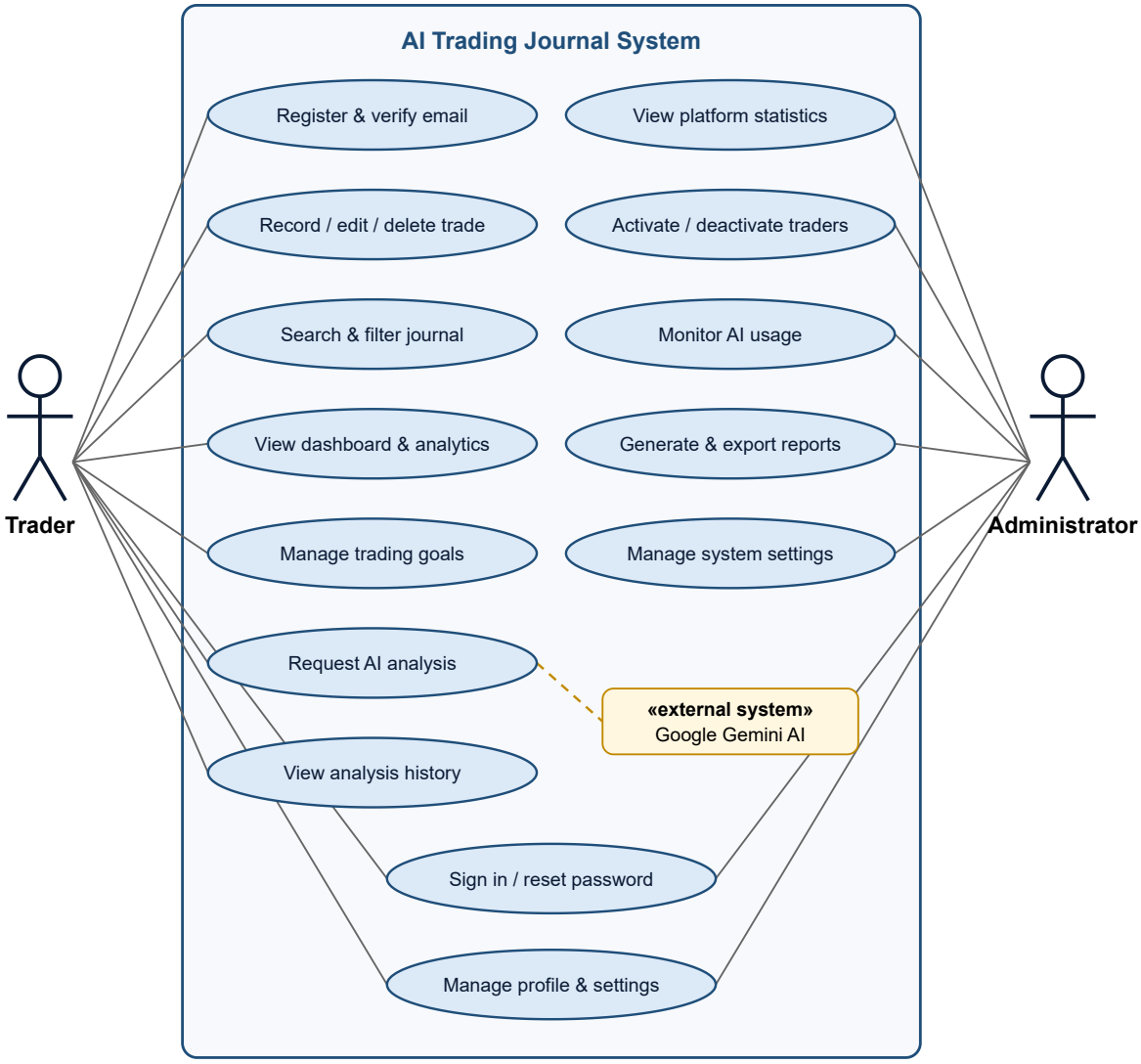


Figure 5.4 Use Case Diagram

5.3.2 Sequence Diagrams

The first sequence diagram shows how an AI performance analysis is produced. Note that the trader's trades are loaded through Row Level Security using the trader's own token, and that the Gemini key is used only inside the Edge Function.

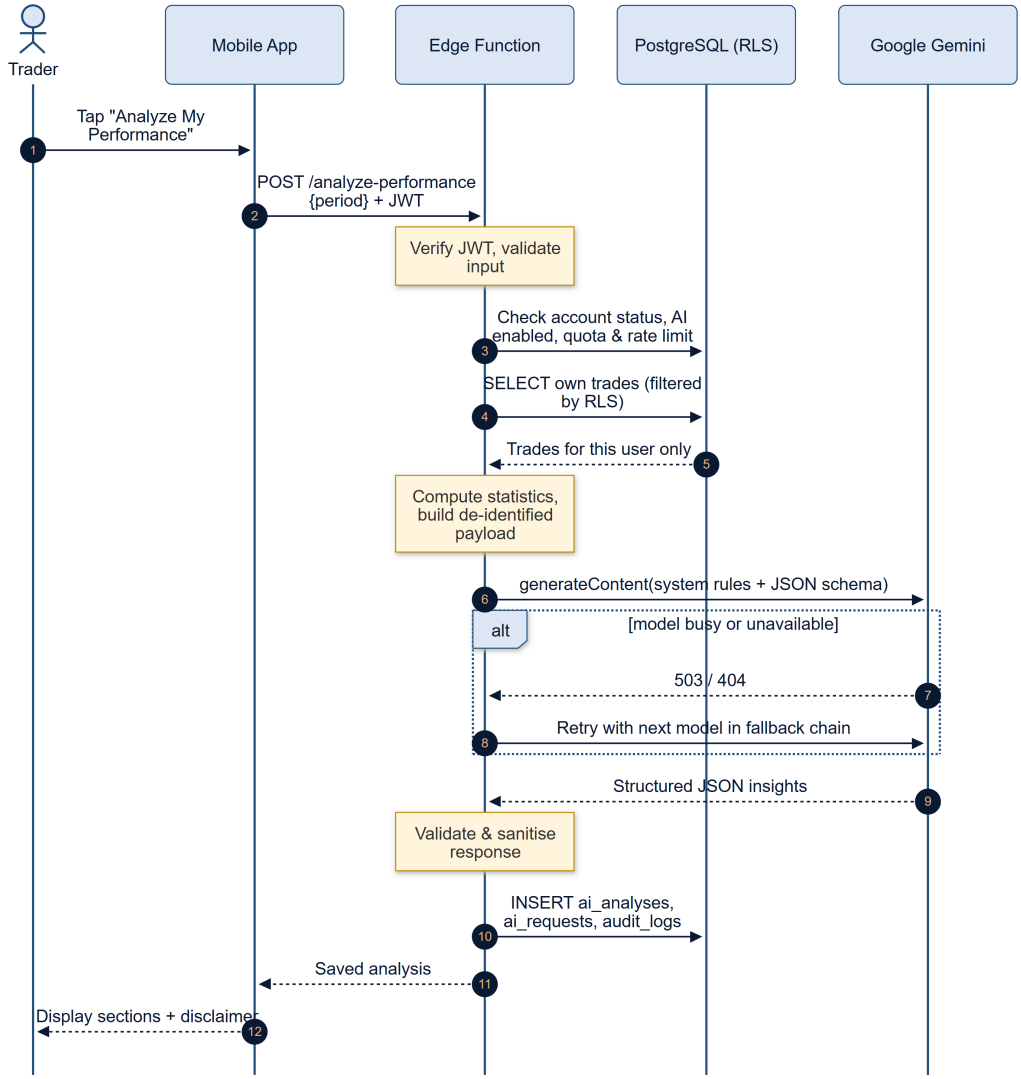


Figure 5.5 Sequence Diagram — AI Performance Analysis

The second sequence diagram shows registration with email verification by one-time code.

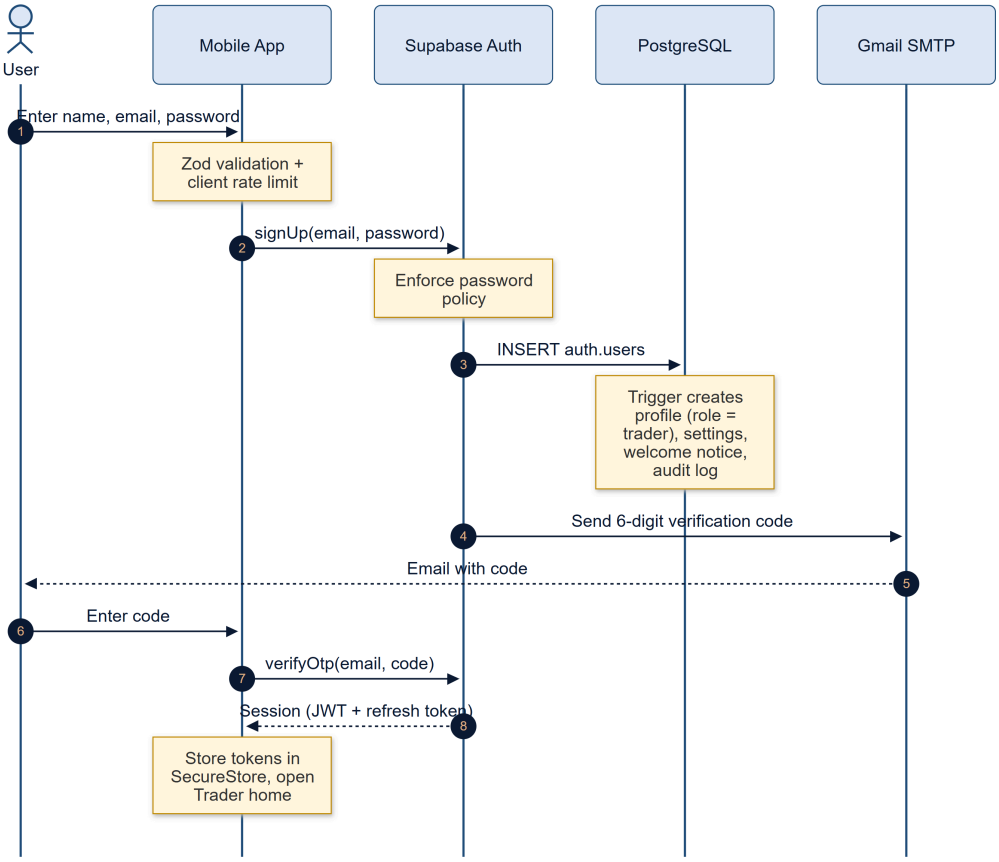


Figure 5.6 Sequence Diagram — Registration and Email Verification

5.3.3 Activity Diagram

The activity diagram shows the main workflow of a trader: recording a trade, reviewing performance and requesting AI insights.

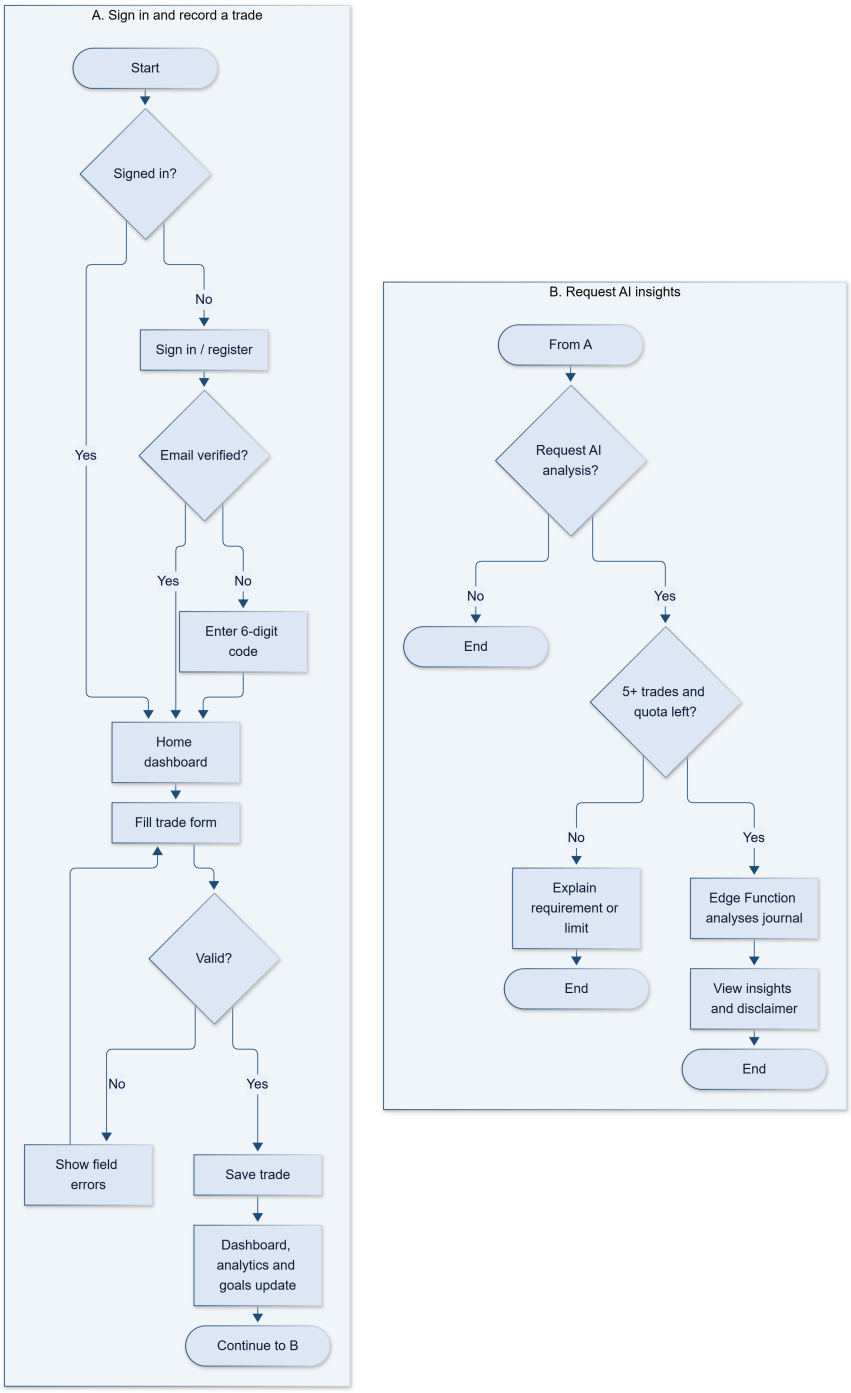


Figure 5.7 Activity Diagram

5.4 Database Schema

The database schema below is generated directly from the deployed Supabase project. It contains 9 tables protected by Row Level Security, 54 CHECK constraints, 25 indexes and 14 triggers. Primary keys are marked **PK**.

profiles	
PK id	uuid
FK user_id	uuid
full_name	text
email	text
role	text
status	text
profile_image	text
trading_experience	text
preferred_sessions	text[]
preferred_pairs	text[]
last_active_at	timestamptz
created_at	timestamptz
updated_at	timestamptz

trades	
PK id	uuid
FK user_id	uuid
currency_pair	text
direction	text
entry_price	numeric
exit_price	numeric
stop_loss	numeric
take_profit	numeric
position_size	numeric
risk_amount	numeric
reward_amount	numeric
risk_reward_ratio	numeric
profit_loss	numeric
trading_session	text
strategy	text
trade_result	text
emotional_state	text
entry_reason	text
notes	text
screenshot_url	text
trade_date	timestamptz
created_at	timestamptz
updated_at	timestamptz

trading_goals	
PK id	uuid
FK user_id	uuid
title	text
description	text
metric	text
target_value	numeric
current_value	numeric
start_date	timestamptz
end_date	timestamptz
status	text
created_at	timestamptz
updated_at	timestamptz

ai_analyses	
PK id	uuid
FK user_id	uuid
analysis_period	text
period_start	timestamptz
period_end	timestamptz
trades_analyzed	integer
scope	text
FK trade_id	uuid
summary	text
analysis_result	jsonb
model	text
created_at	timestamptz

ai_requests	
PK id	uuid
FK user_id	uuid
status	text
scope	text
trades_analyzed	integer
latency_ms	integer
error_code	text
model	text
created_at	timestamptz

notifications	
PK id	uuid
FK user_id	uuid
title	text
message	text
read	boolean
created_at	timestamptz

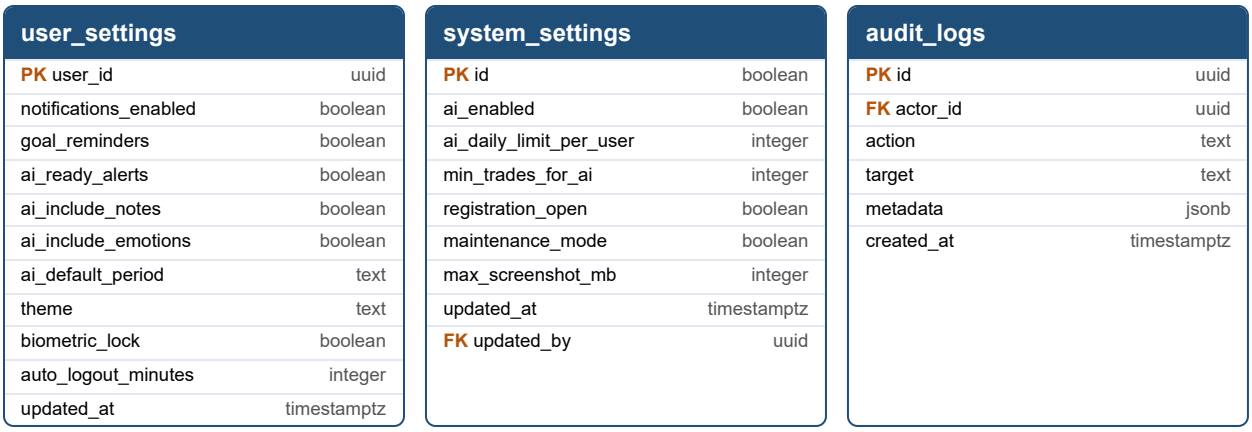


Figure 5.8 Database Schema

5.5 Security Design

Security is enforced in layers so that no single component — least of all the mobile client — is trusted on its own. The table below summarises how each security requirement is met.

Control	Implementation
Authentication	Supabase Auth with bcrypt-hashed passwords, 1-hour JWTs and refresh-token rotation; email verification and password reset by 6-digit code; password policy requiring upper/lower case, digit and symbol.
Token storage	Session tokens stored in the device keychain/keystore (Expo SecureStore) and in sessionStorage on web — never in plain local storage.
Authorisation	Row Level Security on every table (20 policies): traders access only rows where user_id = auth.uid() and only while their account is active.
Least privilege	The anonymous role has no table access; authenticated users may update only specific columns, so role, status and ownership cannot be changed from the app.
Admin privacy	Administrators use SECURITY DEFINER functions that verify the admin role and return aggregates only; raw trades, notes and screenshots are never exposed.
Validation	Zod schemas in the app; CHECK constraints in PostgreSQL (price > 0, stop-loss side, result vs. P/L sign, no future dates, text limits); input validation in the Edge Function.
Rate limiting	Client-side sliding windows; database trigger limiting trade creation to 30 per minute; AI daily quota and 5 requests per hour; provider-level auth limits.
AI security	Gemini key stored only as an Edge Function secret; de-identified payload; journal text treated as data (prompt-injection guard); response validated and sanitised.
Content Security Policy	Web build restricts scripts, connections and images to trusted origins; production headers include HSTS, X-Frame-Options and Permissions-Policy.
Storage	Private screenshot bucket with owner-only folder policies, 5 MB and image-type limits, served through 1-hour signed URLs.
Logging & monitoring	Append-only audit log (sign-up, login, deletes, admin actions, settings, reports, AI calls); structured Edge Function logs; client logger with secret redaction.
Backups & pooling	Managed connection pooling (Supavisor) behind PostgREST and Edge Functions; backups depend on the hosting plan, with manual logical dumps documented.

Table 5.1 Security Controls

5.6 Wireframes

Low-fidelity wireframes were produced before implementation to agree on layout and navigation. They show the sign-in screen, the trader dashboard, the journal, the add-trade form, the AI insights screen and the administrator dashboard.



Figure 5.9 Wireframes

5.7 Summary

This chapter presented the design of the system: a layered architecture that keeps secrets and authorisation on the server, class and entity-relationship models of the domain, use-case, sequence and activity diagrams describing behaviour, the deployed database schema, a layered

security design, and wireframes for the key screens. Together these form a blueprint for the implementation described in the next chapter.

CHAPTER SIX

6. SYSTEM DEVELOPMENT

6.1 Introduction

This chapter describes the implementation of the AI Trading Journal: the tools and technologies used, the resulting user interfaces for traders and administrators, the challenges encountered during development and how they were addressed.

6.2 System Development Tools and Techniques

Tools were selected for cross-platform support, security, low cost and strong documentation. The table below lists the main technologies.

Layer	Technology	Purpose
Language	TypeScript 6	Type-safe code across the application
Mobile framework	React Native 0.86, Expo SDK 57	Android, iOS and web from one codebase; Expo Go for device testing
Navigation	Expo Router	File-based routing with protected route groups per role
UI	React Native Paper, custom SVG charts	Material Design components; line, bar, donut and ranked-bar charts
Forms & validation	React Hook Form, Zod	Typed forms with client-side validation
Data & caching	TanStack React Query	Caching, background refresh, offline pause/resume
Backend	Supabase (PostgreSQL 17, Auth, Storage, Edge Functions)	Database, security policies, authentication, files, server logic
AI	Google Gemini API	Structured performance analysis of journal data
Email	Gmail SMTP	Verification and password-reset codes
Quality	ESLint, TypeScript compiler, Expo Doctor, Puppeteer	Static checks and automated end-to-end testing
Version control	Git, GitHub	Source control and collaboration

Table 6.1 Development Tools and Technologies

6.2.1 Integrated Development Environment (IDE)

Visual Studio Code was used as the main IDE, with TypeScript language support, ESLint integration and an integrated terminal for running Expo, the Supabase CLI and test scripts. The Expo development server allowed the application to be opened instantly on a physical Android phone through Expo Go, with fast refresh after each code change.

Insert screenshot here

Screenshot of Visual Studio Code with the AI Trading Journal project open
(e.g. the mobile/src folder and a screen file).

Save the image as **docs/your-screenshots/vscode.png** and run **npm run build**

Figure 6.1 Visual Studio Code IDE

Insert screenshot here

Photo or screenshot of the app running on your phone through Expo Go,
or the Expo QR code in the terminal.

Save the image as **docs/your-screenshots/expo-go-device.png** and run **npm run build**

Figure 6.2 Application Running on a Physical Device (Expo Go)

6.2.2 Front-end Development

The front end was implemented in TypeScript with React Native and Expo. Screens are organised with Expo Router into an authentication group, a trader group with five bottom tabs (Home, Journal, Analytics, AI Insights, Profile) and an administrator group with six tabs (Dashboard, Users, Data, AI Monitor, Reports, Profile). A single API contract is implemented by a mock client (used in Sprint 1) and a Supabase client (used in production), selected by an environment variable. A custom SVG charting component set was built so that charts render crisply and follow a colour palette validated for colour-blind accessibility.

6.2.3 Backend and Database Management Tools

The Supabase dashboard and Management API were used to create the project, apply versioned SQL migrations, configure authentication (SMTP, email templates, password policy and rate limits), manage storage buckets and deploy the Edge Function. Supabase's security and performance advisors were run after migrations, and the reported issues were fixed.

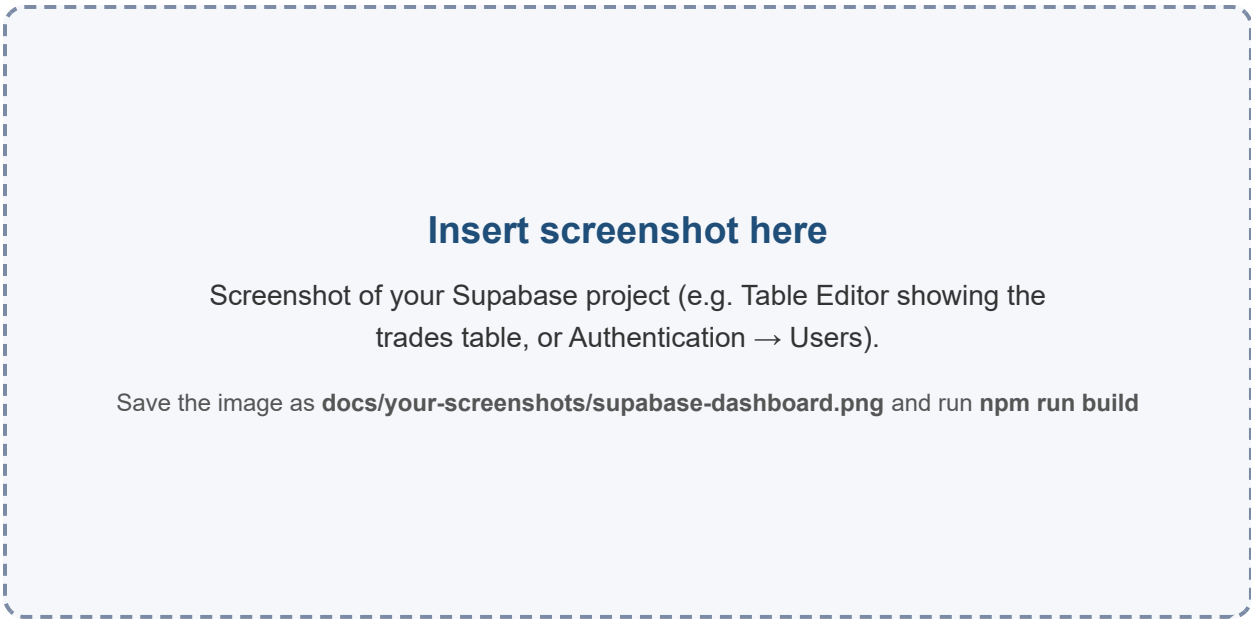


Figure 6.3 Supabase Project Dashboard

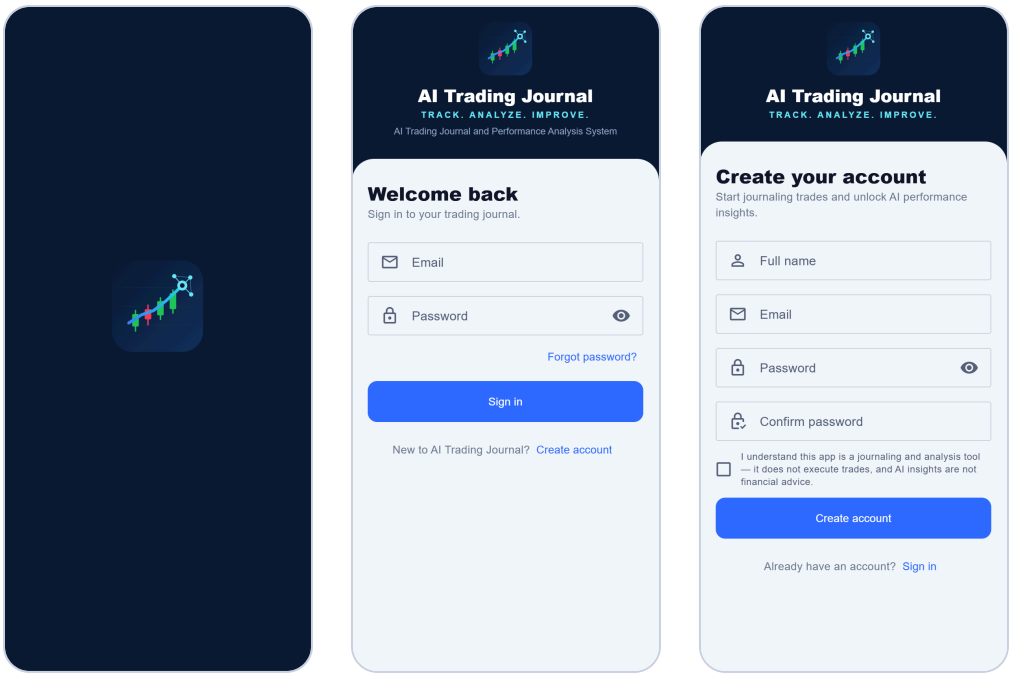
6.2.4 AI Integration

The Gemini API is called only from the analyze-performance Edge Function. The function instructs the model to act as a trading performance analyst, to base observations only on the supplied data, and to return JSON that follows a fixed schema. Because newer Gemini models are occasionally overloaded and older models are withdrawn for new API keys, the function tries a chain of models in order and records any fallbacks for monitoring.

6.3 System Interfaces

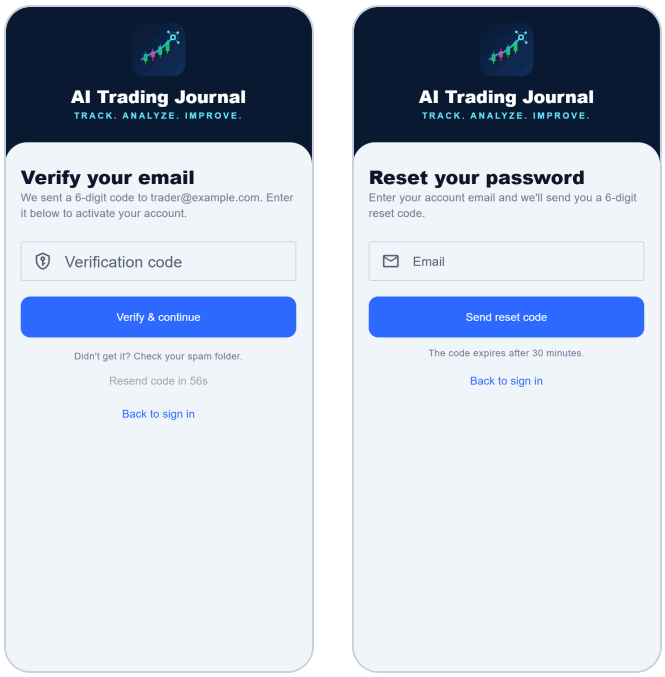
The following figures show the implemented interfaces, captured from the application running against the live backend with demonstration data.

Authentication Interfaces



Splash screen | Sign in | Create account

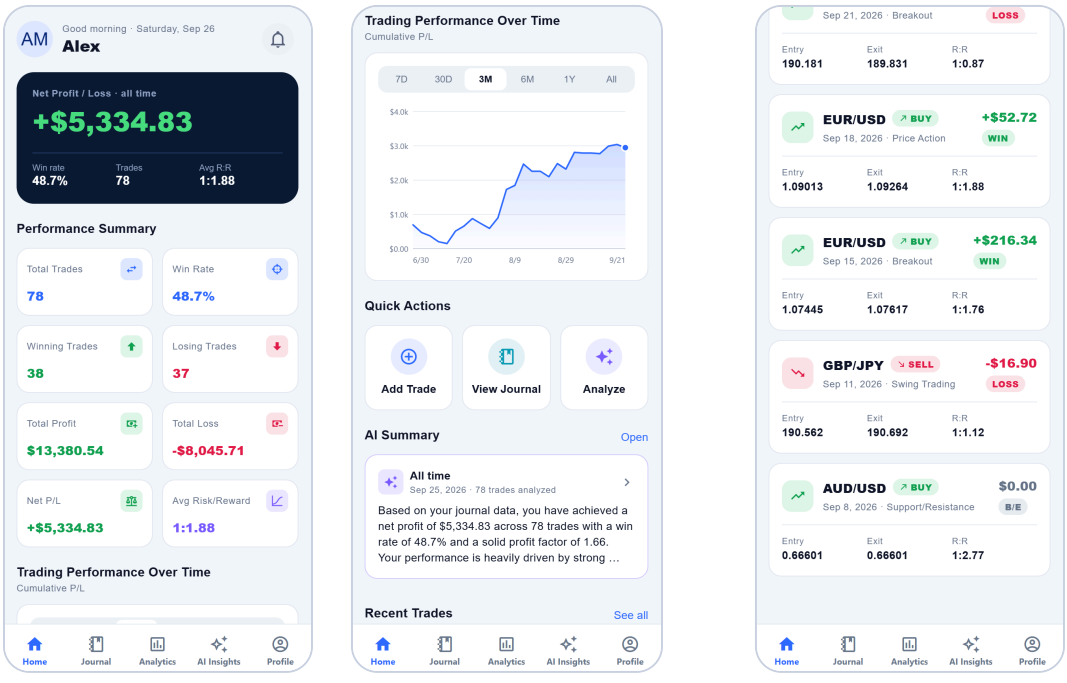
Figure 6.4 Splash, Sign-in and Registration Screens



Verify email (6-digit code) Password reset

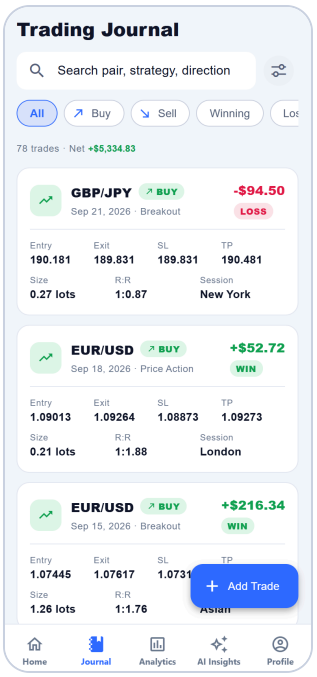
Figure 6.5 Email Verification and Password Reset

Trader Interfaces

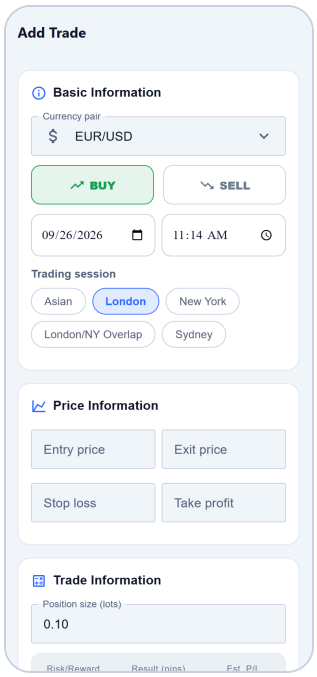


Performance summary Performance chart & quick actions AI summary & recent trades

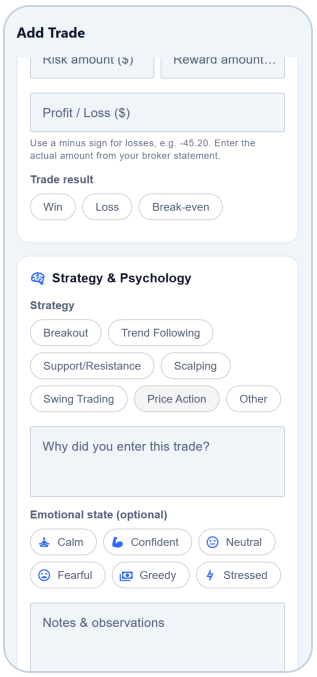
Figure 6.6 Trader Home Dashboard



Journal with search & filters

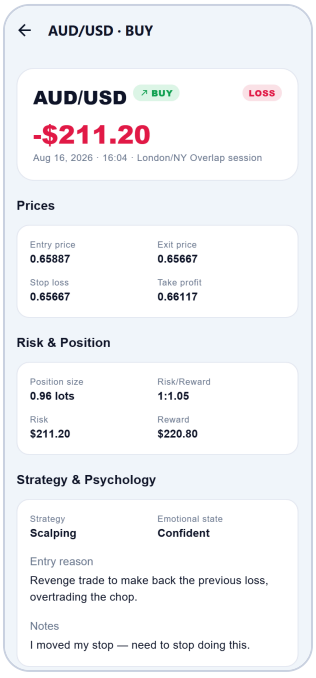


Add trade — basic information

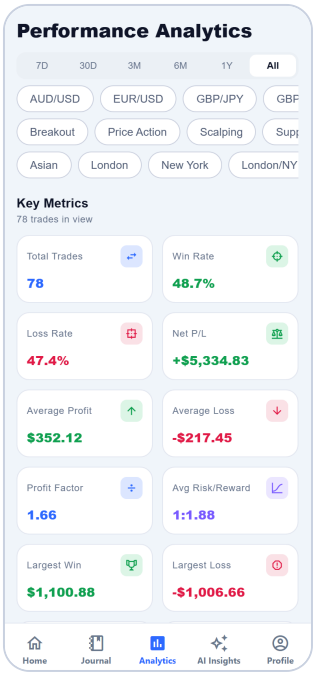


Add trade — prices & risk

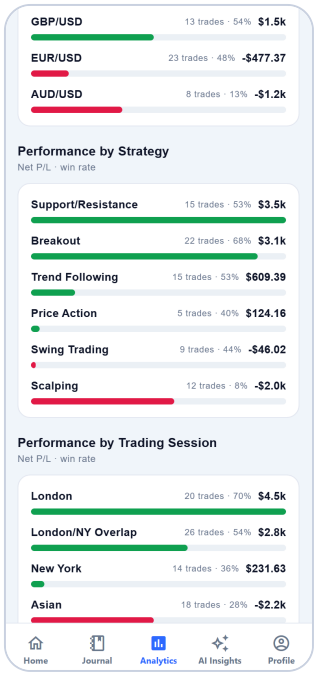
Figure 6.7 Trading Journal and Add Trade



Trade details

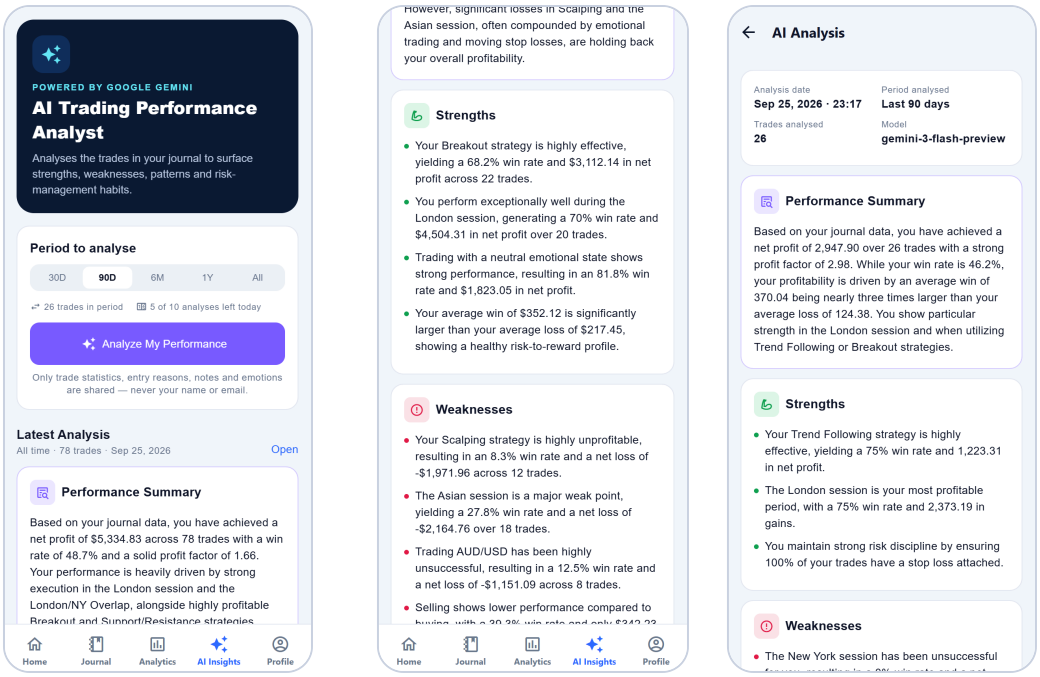


Key metrics



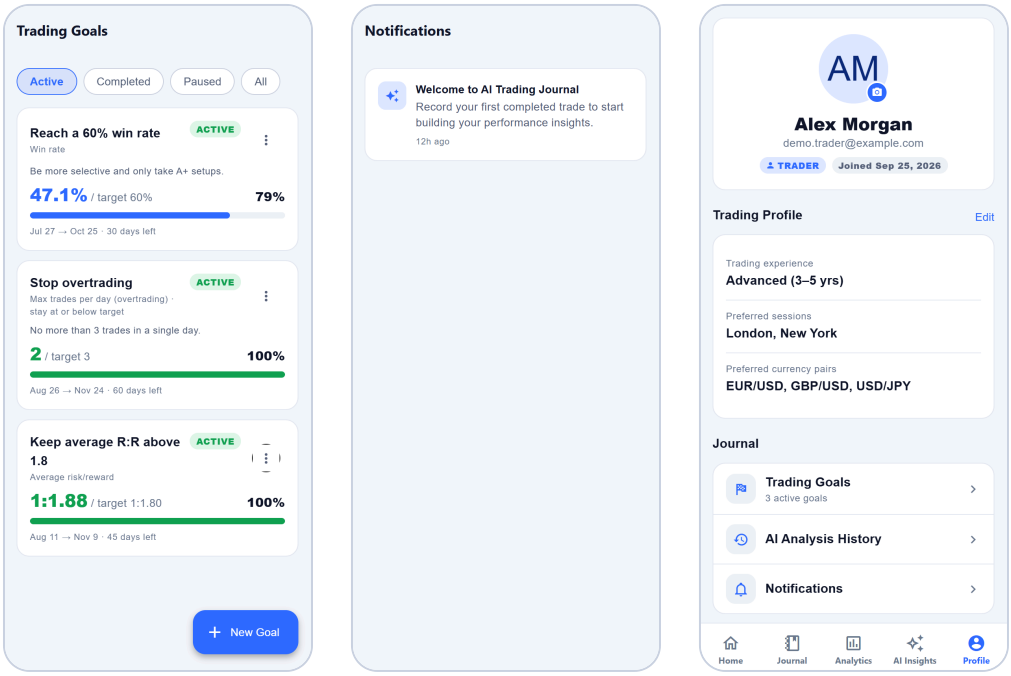
Performance charts

Figure 6.8 Trade Details and Analytics



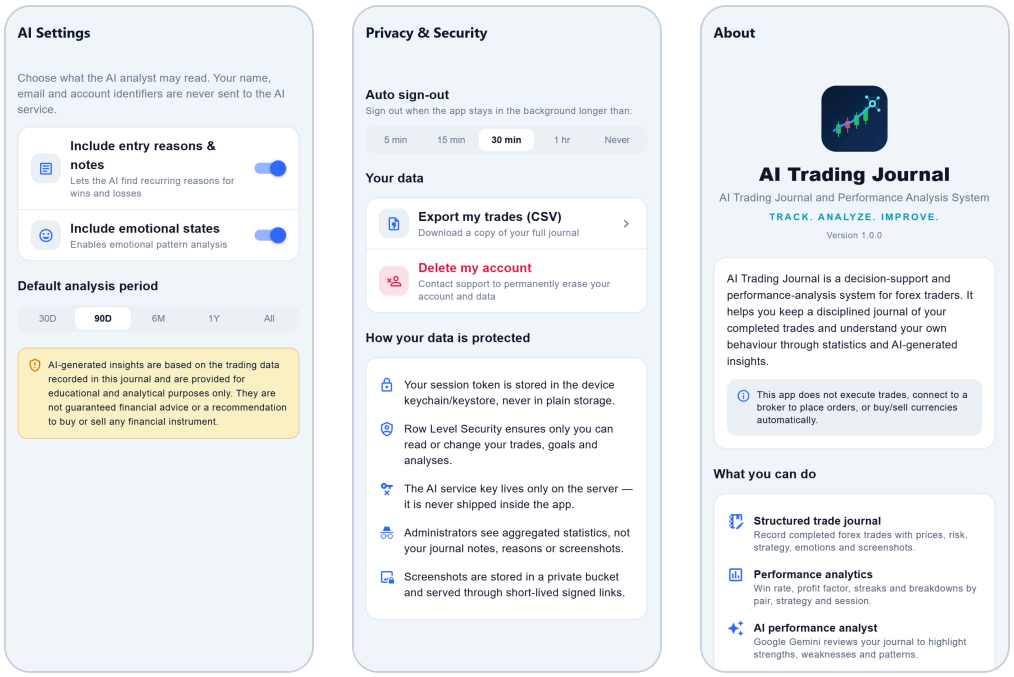
AI Trading Performance Analyst | Gemini analysis result | Saved analysis

Figure 6.9 AI Insights



Trading goals | Notifications | Profile

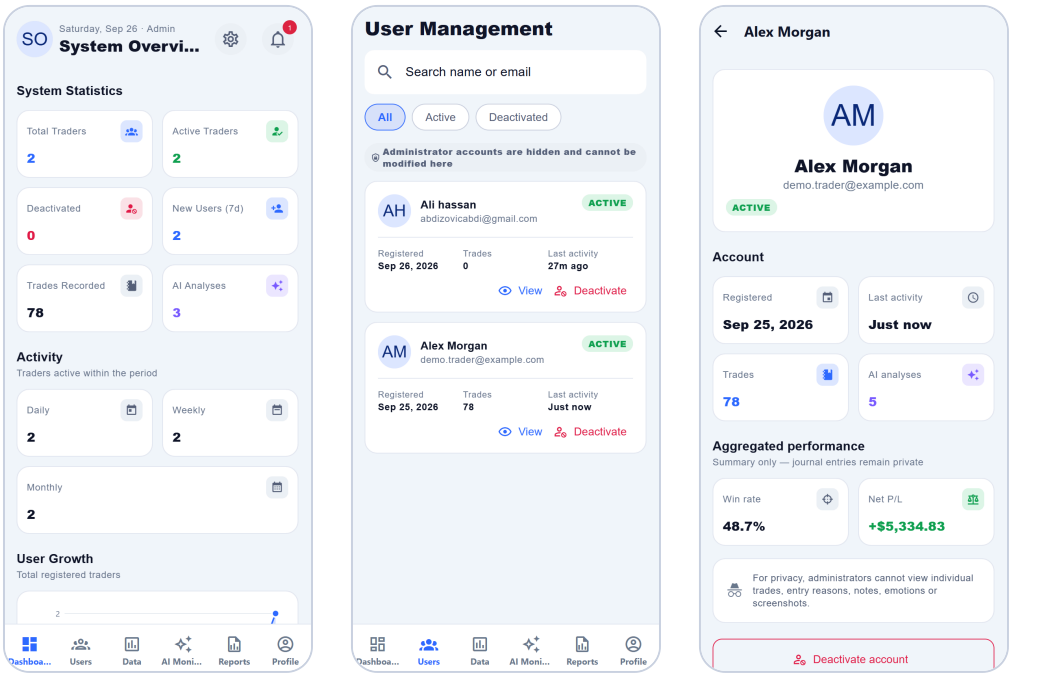
Figure 6.10 Goals, Notifications and Profile



AI data-sharing settings | Privacy & security | About & disclaimer

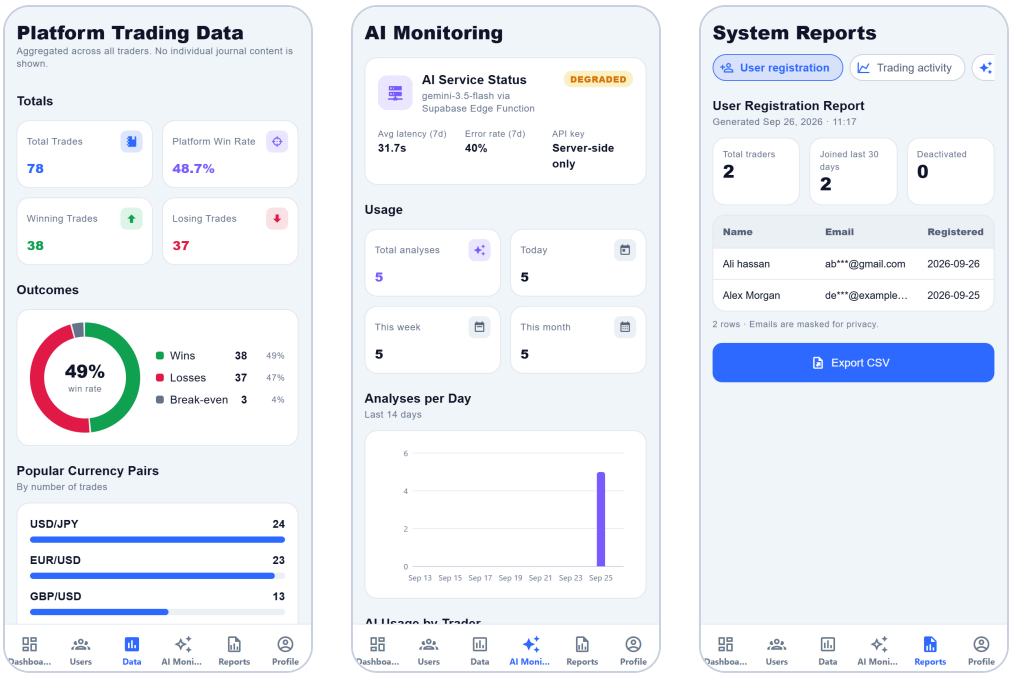
Figure 6.11 Settings and About

Administrator Interfaces



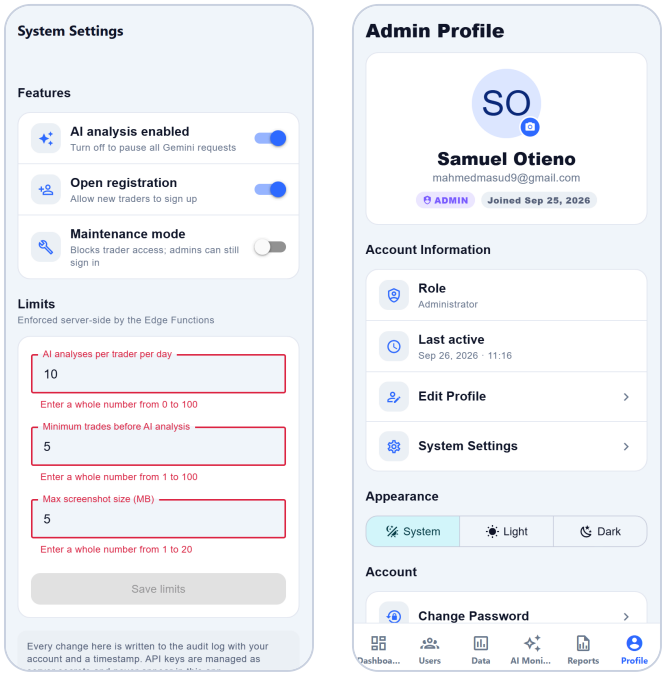
System overview | User management | Trader details (aggregates only)

Figure 6.12 Administrator Dashboard and User Management



Platform trading data AI monitoring Reports & CSV export

Figure 6.13 Administrator Monitoring and Reports



System settings Admin profile & logout

Figure 6.14 Administrator Settings and Profile

6.4 Challenges Encountered During Development

6.4.1 Technical Complexities

Several technical challenges arose. Supabase Auth could not load user records whose ban timestamp was set to “infinity”, which broke sign-in checks for deactivated accounts; this was found by automated tests and fixed by using a far-future date. Session tokens could exceed the size limit of secure device storage, so a chunked storage adapter was implemented. The originally selected Gemini model was withdrawn for new API keys, and newer models were intermittently overloaded; the Edge Function was therefore redesigned with an ordered fallback chain and an overall time budget that respects the platform’s execution limit. Email links do not reliably reopen the app in Expo Go, so verification and password reset were changed to six-digit codes.

6.4.2 User Adoption and Resistance

Traders are often reluctant to spend time journaling. To reduce effort, the trade form suggests the session from the time of day, computes risk/reward and profit or loss automatically, and marks only essential fields as required. Clear empty states guide new users towards their first trade, and AI analysis is unlocked after only five trades so that value is visible early.

6.4.3 Economic Constraints

The project used free and open-source tools and the free tiers of Supabase and the Gemini API. Some protections, such as leaked-password detection, server-side inactivity timeouts and point-in-time recovery, require a paid plan; they were documented for future activation, and equivalent client-side measures such as automatic sign-out after inactivity were implemented.

6.4.4 Continuous Updates and Support

Expo SDKs, React Native and AI models change frequently. Versioned SQL migrations, a typed codebase, automated checks and documented deployment steps make it practical to keep the system up to date, while the model fallback chain protects the AI feature against provider changes.

6.5 Summary

This chapter described the tools and techniques used to implement the AI Trading Journal, presented the trader and administrator interfaces, and discussed technical, adoption, economic and maintenance challenges together with the solutions applied.

CHAPTER SEVEN

7. SYSTEM TESTING AND DEPLOYMENT

7.1 Introduction

Testing verifies that the system meets its functional, non-functional and security requirements, while deployment makes it available to users. This chapter describes the tests carried out — including automated tests run against the live backend — their results, and the current and planned deployment methods.

7.2 System Testing and Validation

Testing was carried out at several levels: static analysis on every change, module-level checks of individual rules, functional end-to-end tests of complete user journeys, user-interface review, and a dedicated security test suite.

7.2.1 Unit and Module Tests

Module tests verified individual components and rules in isolation, particularly validation rules enforced by the database, authentication service and Edge Function.

ID	Module	Test Case	Expected Result	Actual Result	Status
UT01	Sign-up trigger	New user created with role “admin” in metadata	Profile created with role = trader	As expected	**Pass**
UT02	Password policy	Sign up with “password1”	Rejected as weak password	As expected	**Pass**
UT03	Trade constraints	BUY with stop loss above entry	Rejected (trades_sl_side)	As expected	**Pass**
UT04	Trade constraints	Loss with positive P/L	Rejected (result vs P/L)	As expected	**Pass**
UT05	Trade constraints	Trade dated in the future	Rejected	As expected	**Pass**
UT06	Trade constraints	Malformed currency pair	Rejected	As expected	**Pass**
UT07	Edge Function	Invalid analysis period	HTTP 400 validation error	As expected	**Pass**
UT08	Edge Function	Call without a session	HTTP 401 rejected	As expected	**Pass**

Table 7.1 Unit and Module Testing

7.2.2 Functional Testing

Functional tests exercised complete user journeys in an automated browser at phone size against the live backend.

ID	Feature	Test Scenario	Expected Outcome	Actual Outcome	Status
FT01	Sign in	Wrong password	“Incorrect email or password” shown	As expected	**Pass**
FT02	Role routing	Trader signs in	Trader home with 78 journal trades	As expected	**Pass**
FT03	Analytics	Open Analytics tab	Metrics computed from live data	As expected	**Pass**
FT04	Add trade	Complete and save trade form	Journal shows 79 trades	As expected	**Pass**
FT05	Delete trade	Delete with confirmation	Trade removed and audit logged	As expected	**Pass**
FT06	Goals	Open goals	Progress computed from trades	As expected	**Pass**
FT07	AI analysis	Analyze 90 days	Structured Gemini insights saved	As expected	**Pass**
FT08	AI limits	Exceed hourly AI limit	Request refused (HTTP 429)	As expected	**Pass**
FT09	Admin	Admin signs in	Dashboard, users, data, AI, reports load	As expected	**Pass**
FT10	Route guard	Admin opens trader journal URL	Redirected to admin area	As expected	**Pass**

Table 7.2 Functional Testing

7.2.3 User Interface (UI) Testing

Every screen was reviewed at phone size in light and dark themes. UI testing identified and led to fixes for overlapping chart labels, a detached progress-bar layout, a truncated greeting on long names, repeated axis labels on count charts and a hard-to-find administrator logout.

ID	UI Element	Test Scenario	Expected Result	Actual Result	Status
UI01	Sign-in screen	Layout and demo panel visibility	Clean layout; demo panel hidden in production	As expected	**Pass**
UI02	Dashboards	Role-based display	Correct tabs per role	As expected	**Pass**
UI03	Charts	Axis labels and tooltips	No overlapping or repeated labels	As expected	**Pass**
UI04	Forms	Validation feedback	Field-level error messages	As expected	**Pass**
UI05	Themes	Light and dark modes	Readable contrast in both themes	As expected	**Pass**
UI06	States	Loading, empty and error states	Skeletons and helpful messages shown	As expected	**Pass**

Table 7.3 User Interface Testing

7.2.4 Security Testing

A suite of 38 automated security tests was run against the live Supabase project using temporary accounts that were deleted afterwards. The tests attempted to break the access rules as a different trader, as an anonymous client and as an administrator.

Area	Checks	Examples
Cross-user isolation	5	Trader B cannot read, update, delete or create trades belonging to trader A
Privilege escalation	6	Cannot promote self to admin, change own status, reassign ownership, forge AI analyses, call admin functions or change system settings
Anonymous access	4	Anonymous clients cannot read trades or profiles or call admin functions through SQL or REST
Data validation	4	Database rejects wrong-side stop loss, mismatched result, future dates and malformed pairs
Administrator privacy	7	Admin reads aggregates and reports but cannot read raw trade rows; cannot deactivate admins
Account lifecycle	8	Sign-up trigger, deactivation bans sign-in and revokes access immediately, audit logging, REST isolation
Setup & teardown	4	Profile and settings creation; allowed profile updates; own-trade insert

Table 7.4 Security Test Coverage

```
PASS  trader B cannot read trader A's trade
PASS  trader B cannot update trader A's trade
PASS  trader B cannot delete trader A's trade
PASS  trader cannot promote self to admin (column privilege)
PASS  trader cannot read audit_logs
PASS  trader cannot forge AI analyses
PASS  trader cannot call admin functions
PASS  anonymous cannot read trades
PASS  DB rejects BUY with stop loss above entry
PASS  admin cannot read raw trade rows (privacy)
PASS  admin cannot deactivate an admin/self
PASS  deactivated trader cannot sign in (clean user_banned error)
PASS  deactivated trader loses data access immediately
PASS  REST API with B's token returns none of A's trades
   … 24 further checks

38 passed, 0 failed (test users deleted)
```

Figure 7.1 Security Test Suite Output (live backend)

7.3 Testing Results

All test cases passed after the defects found during testing were fixed. Notably, the security suite initially revealed that deactivated accounts caused an authentication error instead of a clean rejection; after the fix, the test was tightened to require the correct "user_banned" response.

Testing Type	Total Test Cases	Passed	Failed	Success Rate
Unit / Module Testing	8	8	0	100%
Functional Testing	10	10	0	100%
UI Testing	6	6	0	100%
Security Testing	38	38	0	100%

Table 7.5 Testing Results

7.4 System Deployment

The backend is deployed on Supabase Cloud (region eu-central-1). Database changes are applied as versioned SQL migrations; storage buckets, authentication settings and the Edge Function are configured and deployed through the Supabase Management API and CLI. The Gemini API key and other secrets are stored as Edge Function secrets. The mobile application reads only the public project URL and anon key from its environment configuration.

7.4.1 Current Deployment Methods

- **Expo Go:** testers install Expo Go (SDK 57) and scan the development server's QR code to run the application on their phones.
- **Web build:** the same codebase is exported as a static web application with a Content Security Policy, suitable for hosting on any static host.
- **Supabase Cloud:** database, authentication, storage and Edge Function are live and shared by all clients.

7.4.2 Future Deployment Plans

- **App stores:** build signed Android (AAB) and iOS binaries with Expo Application Services and publish to Google Play and the Apple App Store.
- **Web hosting:** host the web build with the documented security headers (HSTS, frame protection, CSP).
- **Production hardening:** upgrade the Supabase plan to enable leaked-password protection, session inactivity timeouts, daily backups and point-in-time recovery.

7.5 Summary

This chapter presented the testing and deployment of the AI Trading Journal. Module, functional, user-interface and security tests — many of them automated against the live backend — all passed after the defects they revealed were fixed. The system is deployed on Supabase Cloud and distributed through Expo Go and a web build, with app-store publication planned.

CHAPTER EIGHT

8. DISCUSSIONS, FUTURE WORKS AND RECOMMENDATIONS

8.1 Introduction

This chapter reflects on how the AI Trading Journal met the research objectives, identifies areas for future work, and makes recommendations for the system's long-term success.

8.2 Discussions of Objectives

Objective 1

The study assessed the challenges retail traders face with traditional journaling. Interviews and questionnaires showed inconsistent record keeping, little systematic review, strong emotional influences and weak risk management, consistent with behavioural-finance literature. The system addresses these by making journaling fast, capturing behavioural context (strategy, session, reasons and emotions) and computing the metrics that reveal risky behaviour.

Objective 2

The review of existing tools found that they provide statistics but are often desktop-oriented, subscription-based, weak at capturing context and limited in plain-language interpretation. The developed system is mobile-first, captures context by design and adds AI-generated, privacy-preserving insights with explicit user control over data sharing.

Objective 3

The system was implemented with React Native (Expo SDK 57) and TypeScript on the front end, Supabase for the database, authentication, storage and server logic, and Google Gemini for analysis. Security requirements — validation, rate limiting, content security policy, caching, monitoring, database-level security, authentication, logging and pooling — were implemented across both the client and the backend.

Objective 4

Module, functional, user-interface and security testing confirmed that all core features work as intended. In particular, 38 automated security tests against the live backend confirmed that traders cannot access each other's data, cannot escalate privileges, and that administrators see aggregates only.

8.3 Future Work

8.3.1 Deployment to App Stores

Publishing signed builds to Google Play and the Apple App Store would provide easier installation, automatic updates and greater user trust.

8.3.2 Broker Statement Import

Importing trades from broker statements (for example CSV or MetaTrader history exports) would reduce manual entry while still allowing traders to add reasons and emotions.

8.3.3 Push Notifications and Reminders

Device push notifications could remind traders to journal after trading sessions and alert them when goals are achieved or AI analyses are ready.

8.3.4 Deeper AI Analysis

Future versions could compare analyses over time to show whether identified weaknesses are improving, and allow traders to ask follow-up questions about their results.

8.3.5 Localisation and Accessibility

Adding Swahili and other languages, and further accessibility improvements such as larger text options, would widen adoption.

8.4 Recommendations

8.4.1 Continuous User Engagement

Regular feedback from traders and mentors should guide improvements so that the system continues to meet real needs.

8.4.2 Strengthening Data Security

Before large-scale use, the backend should be moved to a paid plan to enable leaked-password protection, inactivity timeouts and point-in-time recovery, and credentials used during development should be rotated.

8.4.3 Responsible Use of AI

AI insights must continue to be presented as educational analysis of past behaviour, never as trading signals or guaranteed advice, in line with regulatory expectations.

8.4.4 Regular System Updates

Dependencies, Expo SDK versions and AI models should be updated regularly, and the automated test suites re-run after every change.

8.4.5 Collaboration with Trading Communities and Regulators

Partnering with trading communities, educators and, where appropriate, the Capital Markets Authority could promote disciplined, well-informed retail trading.

8.5 Summary

The AI Trading Journal met its objectives by combining fast mobile journaling, automatic analytics and secure, privacy-preserving AI insights in one system. Future work on app-store deployment, broker import, notifications and deeper AI analysis, together with the recommendations above, will strengthen its long-term impact.

APPENDIX

A.1: User Survey Questionnaire

Section A: Retail Trader Questionnaire

Question 1: How do you currently record your trades?
☐ Do not record ☐ Notebook / paper ☐ Spreadsheet ☐ Dedicated app

Question 2: How often do you review your past trading performance?
☐ Never ☐ Rarely ☐ Monthly ☐ Weekly

Question 3: Do emotions (fear, greed, stress) affect your trading decisions?
☐ Often ☐ Sometimes ☐ Rarely

Question 4: Would AI-generated insights on your own journal help you improve?
☐ Strongly agree ☐ Agree ☐ Neutral ☐ Disagree

Section B: Trading Mentor Questionnaire

Question 1: How do you currently review your students' trading records?
☐ Screenshots on WhatsApp ☐ Shared spreadsheets ☐ Verbal discussion only ☐ Other: _____

Question 2: What is the most common weakness you observe in retail traders?
☐ Poor risk management ☐ Overtrading ☐ Emotional trading ☐ Other: _____

Section C: Community Administrator Questionnaire

Question 1: Would aggregated platform statistics help you support your community?
☐ Yes ☐ Not sure ☐ No

A.2: Work Plan and Schedule

Task	Start Date	End Date	Duration	Description
Requirements gathering	2026-06-01	2026-06-12	2 weeks	Interviews, questionnaires, literature and competitor review
System design	2026-06-15	2026-06-26	2 weeks	Architecture, UML diagrams, database schema, wireframes
Sprint 1 — Front end (mock API)	2026-06-29	2026-07-24	4 weeks	Branding, navigation, trader and admin screens, charts, forms
Sprint 2 — Backend	2026-07-27	2026-08-14	3 weeks	Supabase schema, constraints, RLS, storage, auth configuration
Sprint 3 — AI integration	2026-08-17	2026-08-28	2 weeks	Gemini Edge Function, quotas, validation, fallback
Sprint 4 — Integration & hardening	2026-08-31	2026-09-18	3 weeks	Live backend, email verification, security fixes
System testing	2026-09-21	2026-09-30	1.5 weeks	Module, functional, UI and security testing
Documentation & presentation	2026-10-01	2026-10-16	2.5 weeks	Project report, screenshots, final presentation

Table A.1: Work Plan and Schedule

A.3: Budget

Item	Quantity	Cost (KES)	Total (KES)	Notes
Android test smartphones (low and mid range)	2	18,000	36,000	Testing across device sizes
Internet data bundles	4 months	3,000	12,000	Development, testing and backend access
Supabase Pro plan (optional)	4 months	3,300	13,200	Backups, leaked-password protection, session controls
Google Gemini API usage	—	—	5,000	Pay-as-you-go beyond the free tier
Google Play developer registration	1	3,300	3,300	One-time fee for publishing
Laptop maintenance & tools	1	8,000	8,000	Hardware upkeep; development tools are free
Transport & data collection	—	—	6,000	Interviews with traders and mentors
Printing, binding & documentation	—	—	4,000	Final report copies
Contingency	—	—	8,000	Unforeseen expenses
Total Estimated Budget			**95,500**	

Table A.2: Budget

A.4: My Project Repository

Source code: **https://github.com/your-username/ai-trading-journal**

Insert screenshot here

Screenshot of your GitHub repository page for the AI Trading Journal project.

Save the image as **docs/your-screenshots/github-repository.png** and run **npm run build**

A.5: Plagiarism Report

Insert screenshot here

Screenshot or first page of your plagiarism (similarity) report.

Save the image as **docs/your-screenshots/plagiarism-report.png** and run **npm run build**

REFERENCES

Bank for International Settlements. (2022). *Triennial Central Bank Survey of foreign exchange and over-the-counter (OTC) derivatives markets in 2022*. BIS.

Barber, B. M., & Odean, T. (2000). Trading is hazardous to your wealth: The common stock investment performance of individual investors. *The Journal of Finance, 55*(2), 773–806.

Barber, B. M., & Odean, T. (2001). Boys will be boys: Gender, overconfidence, and common stock investment. *The Quarterly Journal of Economics, 116*(1), 261–292.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., et al. (2001). *Manifesto for Agile Software Development*. https://agilemanifesto.org

Capital Markets Authority. (2017). *The Capital Markets (Online Foreign Exchange Trading) Regulations, 2017*. Republic of Kenya.

Douglas, M. (2000). *Trading in the zone*. Prentice Hall Press.

Elder, A. (1993). *Trading for a living: Psychology, trading tactics, money management*. John Wiley & Sons.

European Securities and Markets Authority. (2018). *ESMA agrees to prohibit binary options and restrict CFDs to protect retail investors* (ESMA71-98-128). ESMA.

Expo. (2026). *Expo documentation (SDK 57)*. https://docs.expo.dev

Google. (2026). *Gemini API documentation*. https://ai.google.dev/gemini-api/docs

ISO/IEC 25010. (2011). *Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

Kahneman, D., & Tversky, A. (1979). Prospect theory: An analysis of decision under risk. *Econometrica, 47*(2), 263–291.

Lo, A. W., Repin, D. V., & Steenbarger, B. N. (2005). Fear and greed in financial markets: A clinical study of day-traders. *American Economic Review, 95*(2), 352–359.

Meta Platforms. (2026). *React Native documentation*. https://reactnative.dev

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

Odean, T. (1998). Are investors reluctant to realize their losses? *The Journal of Finance, 53*(5), 1775–1798.

OWASP Foundation. (2021). *Application Security Verification Standard 4.0.3*. https://owasp.org/www-project-application-security-verification-standard/

OWASP Foundation. (2024). *OWASP Mobile Top 10*. https://owasp.org/www-project-mobile-top-10/

Pressman, R. S., & Maxim, B. R. (2015). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org

Shefrin, H., & Statman, M. (1985). The disposition to sell winners too early and ride losers too long: Theory and evidence. *The Journal of Finance, 40*(3), 777–790.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

Steenbarger, B. N. (2003). *The psychology of trading: Tools and techniques for minding the markets*. John Wiley & Sons.

Supabase. (2026). *Supabase documentation: Database, Auth, Storage, Edge Functions and Row Level Security*. https://supabase.com/docs

Tharp, V. K. (2007). *Trade your way to financial freedom* (2nd ed.). McGraw-Hill.